



Middle Technical University
Technical Engineering College of Artificial Intelligence
Department of Cybersecurity Technology Engineering
Third Year

PYTHON PROGRAMMING

Lecture 1: Introduction

By: Ass. Lec. Abbas Aqeel Kareem

Lecture 1: Introduction

This Lecture prepares you to learn how to program with Python. Preparation includes a brief introduction to computers and computing, programming, and programming languages, as well as the installation of Python and Python interactive programming and integrated development environments (IDEs)

Learning Objectives

After completing this chapter, you should be able to

- Discuss computer programming languages.
- Describe Python and discuss its development, features, and advantages.
- Install Python and required Python IDEs on the computer.
- Get Python and Python IDEs running for the learning activities included in this textbook.

1. Computer programs

A **computer** is an electronic device that stores and processes information. Examples of computers include smartphones, tablets, laptops, desktops, and servers. Technically, a **program** is a sequence of instructions that a computer can run. Programs help people accomplish everyday tasks, create new technology, and have fun.

One of the key principles of modern computers is using stored programs, which are sequences of instructions telling computers what to do. The task of writing programs for computers is called programming.

During the first generation of computers, **machine languages** and assembly languages were used to write programs. Machine languages use sequences of 0 and 1 to code instructions for computers. Programs in a machine language can be directly understood by the CPU of the target computer. The following is an example of machine language for the Intel 8088 CPU:

```
00000011 00000100
```

```
10001000 11000011
```

As you can see, although machine language is friendly to computers, it is not friendly to people, because the instruction itself does not tell us what it does in a human- readable language. To solve this problem, **assembly language** was invented. The two instructions above are written in assembly language as follows:

```
ADD AX, [SI]
```

```
MOV BL, AL
```

The instruction in assembly language is much more friendly to people because one can more easily tell what each instruction does. However, assembly language is not as friendly to computers. For computers to understand programs in assembly language, an **assembler** is needed to translate the programs into code in machine language (Figure 1).

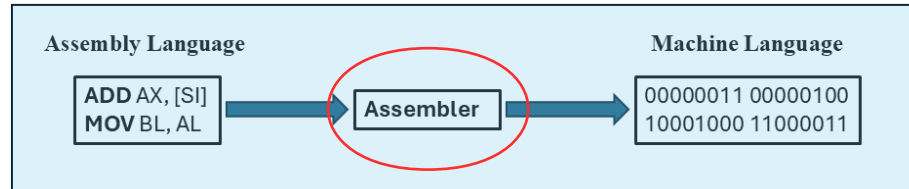
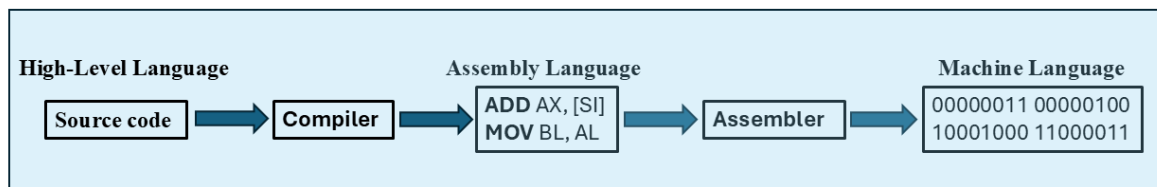


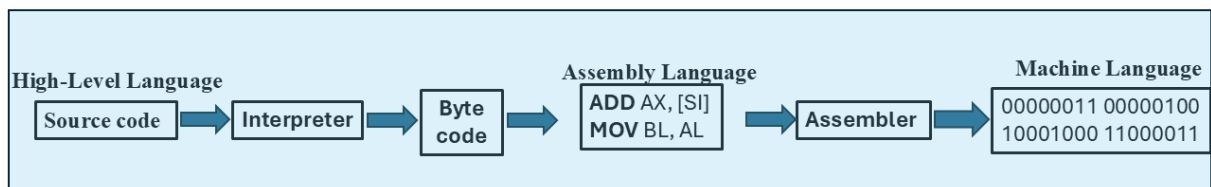
Figure 1: Evolution from machine language to assembly

With assembly language, programming is still a difficult task because instructions in an assembly language are mostly direct mapping of their machine language equivalent and only describe fine and tiny steps of CPU operations. It is difficult to program in assembly language to solve real- world problems.

High- level programming languages are friendly to people but not friendly to computers. For computers to understand and execute programs written in a high- level programming language, they must be translated into machine language. Based on how programs in high- level languages are translated into target machine language, high- level programming languages are either **compiled** or **interpreted** (see Figure 2). A program written in a compiled programming language such as C or C++ needs to be **compiled** into its target machine's codes and linked with copies of code in static libraries, or linked with references to code in dynamic or shared libraries, so that the resulting object can then be executed by the target computer.



(a)



(b)

Figure 2: (a) Compilation process for compiled languages; (b) Interpretation process for interpreted languages.

Programs written in an interpreted programming language can be executed directly by the **interpreter** of the programming language, in view of programmers and users. Behind the scenes, however, to speed up the execution of programs written in an interpreted programming language, the programs are often translated first into some intermediate codes, often called **bytecodes**. The bytecodes are then executed within a so called **virtual machine** built into the interpreter of the programming language.

2. Python Programming Language

2.1. The Development and Implementation of Python

Python is an interpreted, high- level, general- purpose programming language. It is one of the most popular programming languages in the world as shown in figure 3. Python was originally conceived and developed in the Netherlands by Guido van Rossum in the late 1980s. Its first release was Python 0.9.0 in 1991. Python 2.0 was released in 2000, nine years after the release of Python 0.9.0. Most of the development of Python was accomplished between 2005 and 2013, when Guido van Rossum was working at Google, where he spent half of his time on the development of the programming language. The latest Python 2 release is Python 2.7.16, whereas the latest release of Python 3 is Python 3.9.0, at the time of writing. Python is an interpreted programming language rather than a compiled programming language. Programs written in Python don't need to be compiled into target machine code. Instead, they only need to be translated into bytecodes, then executed by a Python Virtual Machine (PVM), which is built into the Python interpreter.

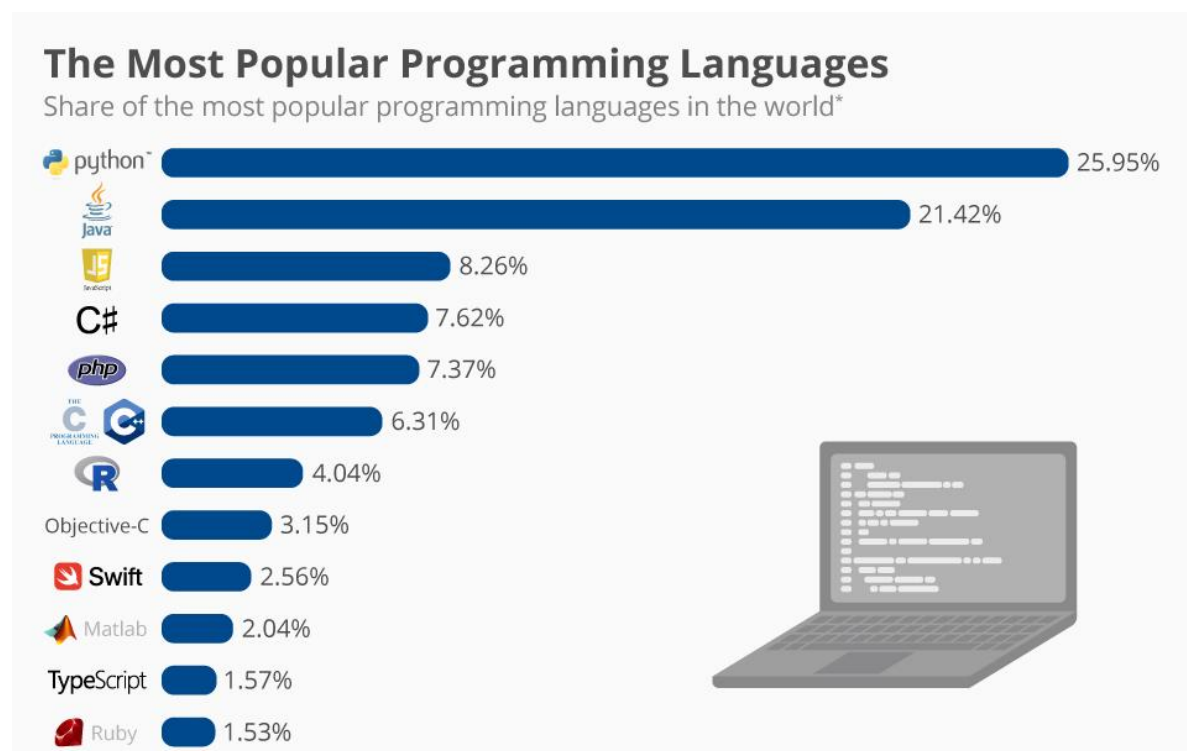


Figure 3: The Most Popular Programming Languages

Python language has different implementations with different names. CPython, written in C and known simply as Python, is what we will be using, and it is the implementation you will get from Python standard distribution at python.org by default. In addition to CPython, the following are some alternative implementations of Python:

1. JPython or Jython, and implementation for running on Java Virtual Machine (JVM)
2. IronPython, an implementation for running on .NET
3. PyPy, an implementation for working with a just-in-time (JIT) compiler
4. Stackless Python, an implementation of a branch of CPython supporting microthreads
5. MicroPython, an implementation for running on microcontrollers

These implementations may be of interest only for some special applications.

2.2. Applications of Python Programming

Python is used in many application domains. Here's a sampling.

1. **Web and Internet Development** : Python offers many choices for [web development](#):
 - Frameworks such as [Django](#) and [Pyramid](#).
 - Micro-frameworks such as [Flask](#) and [Bottle](#).
 - Advanced content management systems such as [Plone](#) and [django CMS](#).
2. **Scientific and Numeric** : Python is widely used in [scientific and numeric](#) computing:
 - [SciPy](#) is a collection of packages for mathematics, science, and engineering.
 - [Pandas](#) is a data analysis and modeling library.
 - [IPython](#) is a powerful interactive shell that features easy editing and recording of a work session, and supports visualizations and parallel computing.
 - The [Software Carpentry Course](#) teaches basic skills for scientific computing, running bootcamps and providing open-access teaching materials.
3. **Education**: Python is a superb language for teaching programming, both at the introductory level and in more advanced courses.
 - Books such as *How to Think Like a Computer Scientist*, *Python Programming: An Introduction to Computer Science*, and *Practical Programming*.
 - The Education Special Interest Group is a good place to discuss teaching issues.
4. **Desktop GUIs**: The Tk GUI library is included with most binary distributions of Python. Some toolkits that are usable on several platforms are available separately:
 - [wxWidgets](#)
 - [Kivy](#), for writing multitouch applications.
 - Qt via [pyqt](#) or [pyside](#)
5. **Software Development**: Python is often used as a support language for software developers, for build control and management, testing, and in many other ways.
 - [SCons](#) for build control.
 - [Buildbot](#) and [Apache Gump](#) for automated continuous compilation and testing.

- [Roundup](#) or [Trac](#) for bug tracking and project management.
- 6. **Business Applications:** Python is also used to build ERP and e-commerce systems:
 - [Odoo](#) is an all-in-one management software that offers a range of business applications that form a complete suite of enterprise management applications.
 - [Tryton](#) is a three-tier high-level general purpose application platform.

2.3. Popular Applications Made With Python

1. Instagram: A major social media platform that utilizes Python for its backend, notably leveraging the Django framework.
2. Spotify: The music streaming giant uses Python extensively for data analysis, recommendation systems, and various backend services.
3. Dropbox: This popular cloud storage service relies on Python for its server-side logic and backend operations.
4. Reddit: The social news aggregation and discussion platform has a significant portion of its codebase in Python.
5. Pinterest: Another prominent social media platform that uses Python for its backend.
6. Uber: The ride-sharing and logistics company utilizes Python for various functionalities, including its backend services.
7. Google: Python is a core language at Google, used in various services like YouTube, Google App Engine, and for internal tools and infrastructure.

2.4. Advantages and limitations

- **Designed for everyone:** Clean, readable, and beginner-friendly syntax.
- **General-purpose and powerful:** Suitable for a wide range of applications.
- **Low overhead:** Simpler than languages like Java (e.g., “Hello World!” in one line).
- **Supports multiple paradigms:** Structured, imperative, object-oriented, functional, and procedural programming.
- **Extensive libraries:** Thousands of third-party packages plus large standard libraries. Examples: NumPy, SciPy, Pandas, Matplotlib, TensorFlow.
- **Flexibility:** Used in fields like data analytics, machine learning, web apps, and automation

While Python offers substantial benefits, it is not without limitations. As an interpreted language, its execution speed is typically slower than that of compiled languages such as C or C++, making it less suitable for applications where real-time performance is critical.

2.5. Install Python

Windows doesn't typically come with a system Python. Fortunately, installation involves little more than downloading and running the Python installer from the Python.org website.

Step 1: Download the Python3 Installer

Open a web browser and navigate to the following URL:
<https://www.python.org/downloads/windows/>

Click Latest Python 3 Release- Python 3.x.x located beneath the “Python Releases for Windows” heading near the top of the page. As of this writing, the latest version was Python 3.11. Then scroll to the bottom and click Windows x86-64 executable in staller to start the download.

Step 2: Run the Installer

Open your Downloads folder in Windows Explorer and double-click the file to run the installer. A dialog that looks like the following one will appear:



It's okay if the Python version you see is greater than 3.9.0 as long as the version is not less than 3. Make sure you select the box that says Add Python3.x to PATH. If you install Python without selecting this box, then you can run the installer again and select it.

Click Install Now to install Python 3. Wait for the installation to finish, then continue to open IDLE.

You can open IDLE in two steps:

1. Click the Start menu and locate the Python 3.9 folder.
2. Open the folder and select IDLE (Python 3.9).

IDLE's interactive window contains a Python shell, which is a textual user interface used to interact with the Python language. You can type a bit of Python code into the interactive window and press Enter to immediately see the results. Hence the name interactive window. The interactive window opens automatically when you start IDLE. You'll see the following text, with some minor differences depending on your setup, displayed at the top of the window:

```
Python 3.9.0 (tags/v3.9.0:1b293b6)
[MSC v.1916 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information
>>>
```

This text shows the version of Python that IDLE is running. You can also see information about your operating system and some commands you can use to get help and view information about Python.

The >>> symbol in the last line is called the prompt. This is where you'll type in your code. Let's try something a little more interesting than adding numbers. A rite of passage for every programmer is writing a program that prints the phrase "Hello, World" on the screen. At the prompt in the interactive window, type the word print followed by a set of parentheses with the text "Hello, World" inside:

```
>>> print("Hello, World")
Hello, World
```

A function is code that performs some task and can be invoked by a name. The above code invokes, or calls, the print() function with the text "Hello, World" as input.

2.6. Program with VS Code IDE

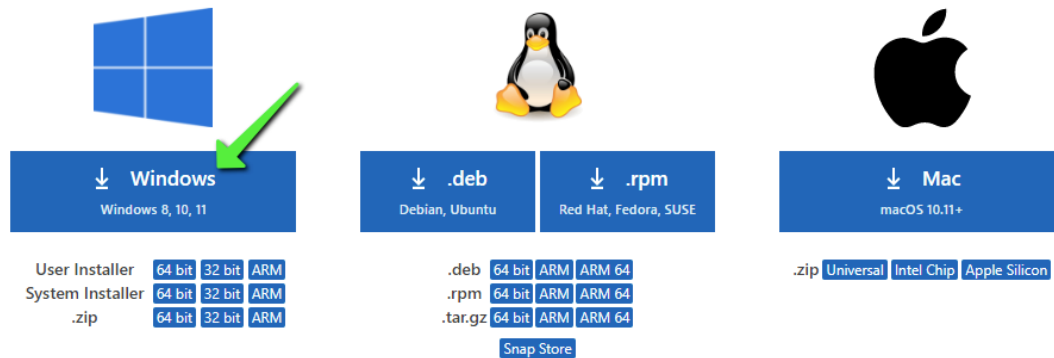
Interactive programming environments like IDLE are good for testing Python statements, code blocks, and some real programming tasks such as data analytics, where interactive programming is more suitable. However, writing programs that contain thousands of lines of code in a Python interactive shell or Jupyter Notebook is inconvenient. As previously mentioned, we will be using VS Code as our IDE. **Installing Visual Studio Code on Windows:**

Step 1: Downloading Visual Studio Code

Navigate to the following link: <https://code.visualstudio.com/download> Click on the blue download button that says Windows under the Windows Logo:

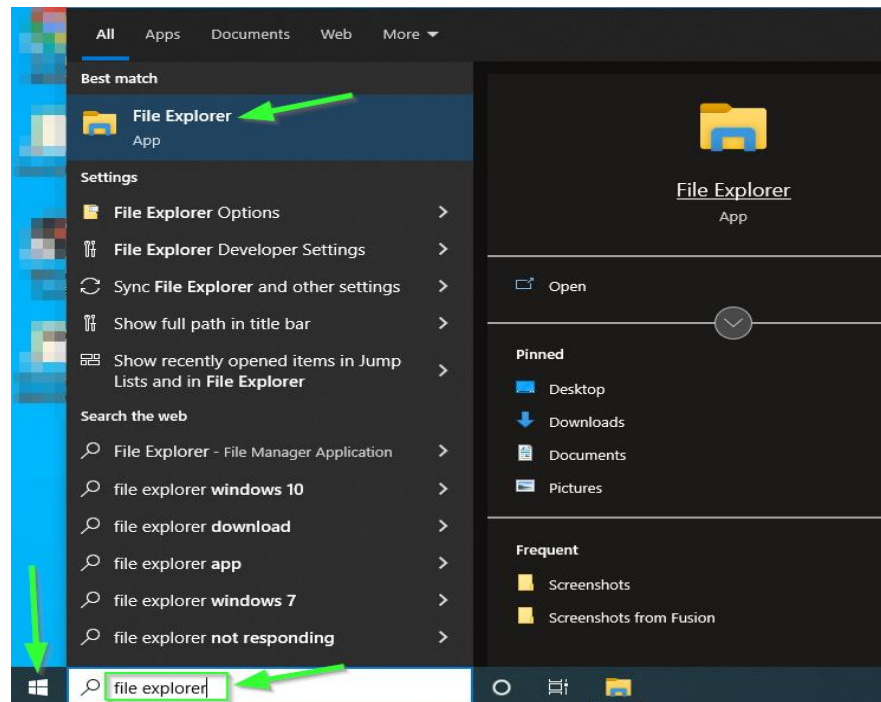
Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



Step 2: Launching the Visual Studio Code Installation Package

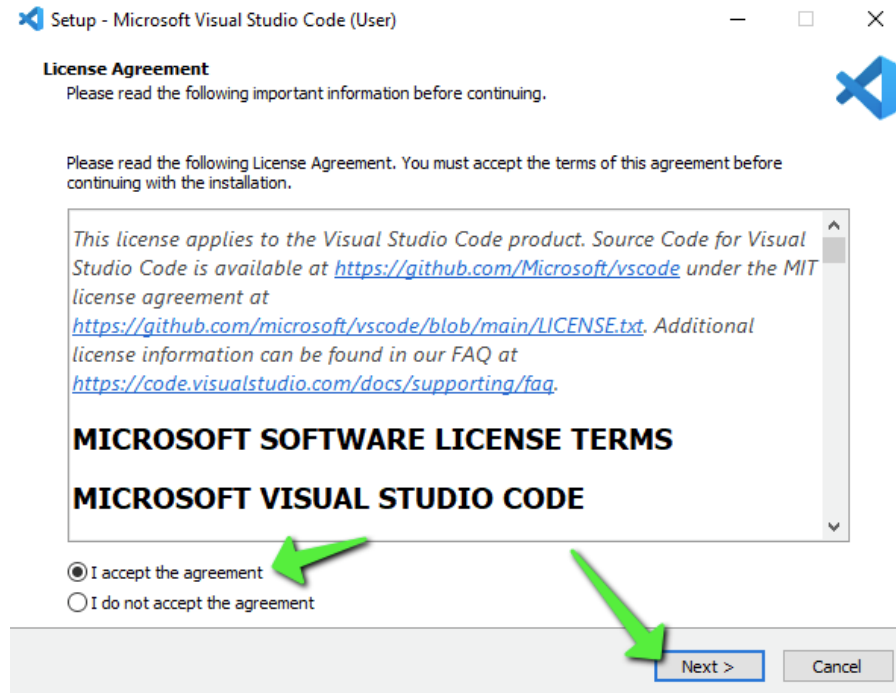
Once the download completes, locate it by clicking on the Start Button and type 'File Explorer'. Launch File Explorer:



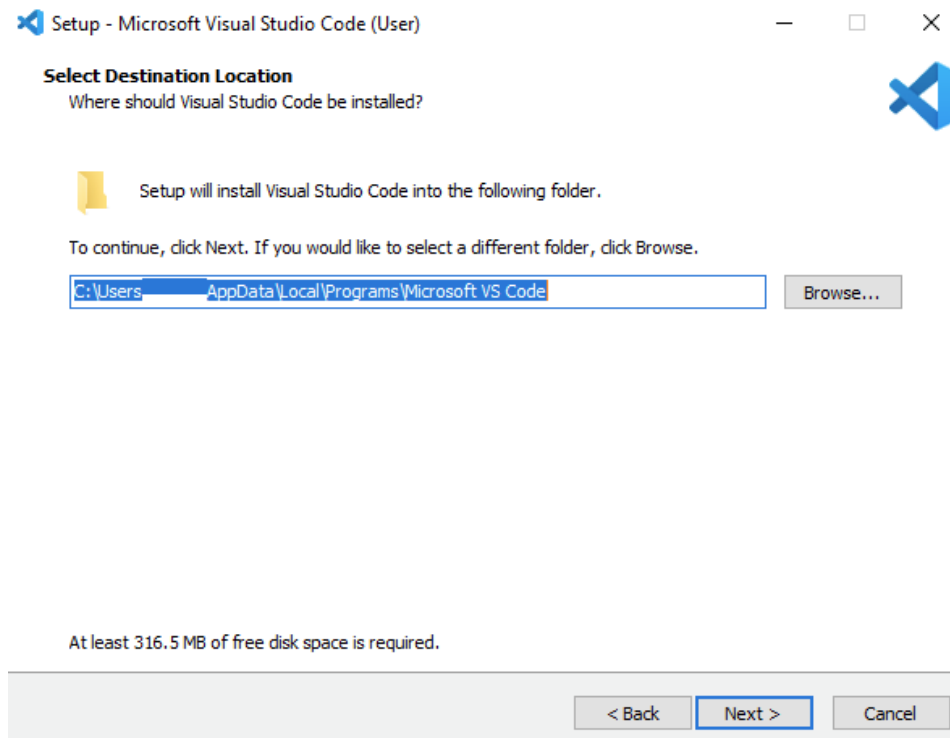
On the left side of the File Explorer window, locate and click on Downloads. Double-click on the downloaded 'VSCodeUserSetup-x64-1.x.x.exe' file.

Step 3: Installing

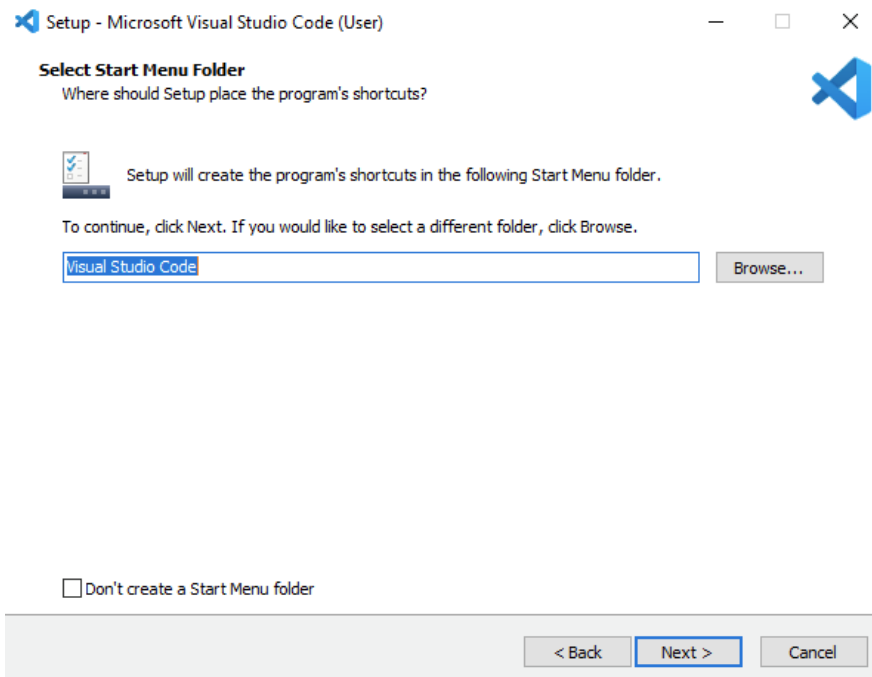
Once the installer launches, step through the installation process. First, accept the License Agreement, then click Next .



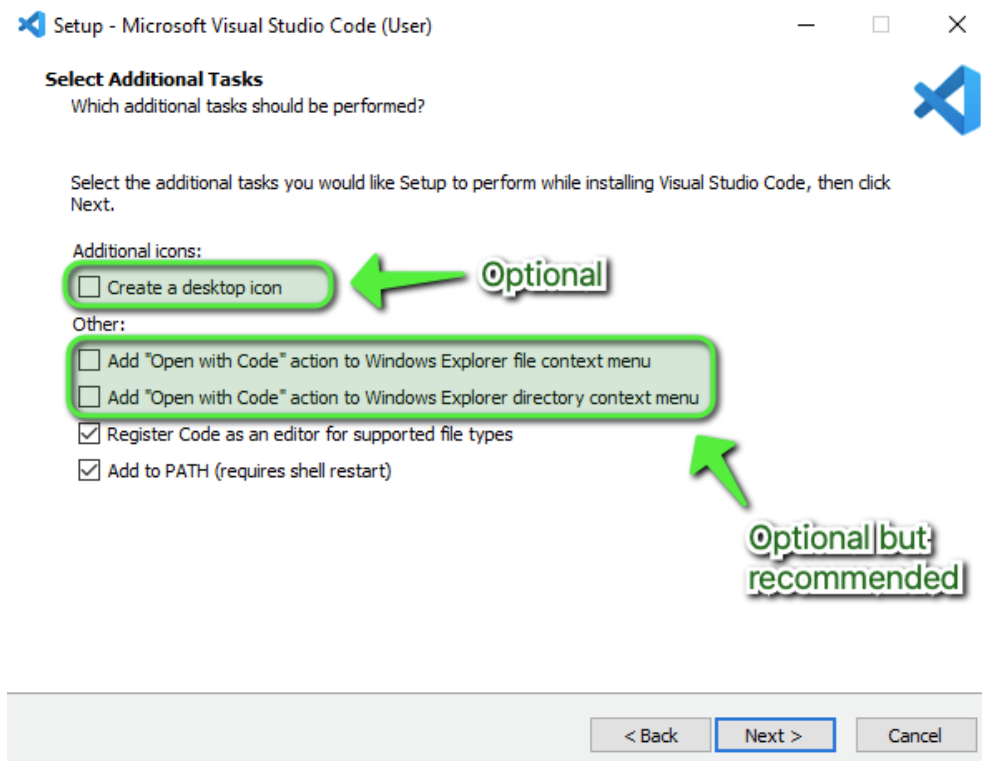
Accept the default location for installation by clicking Next >.



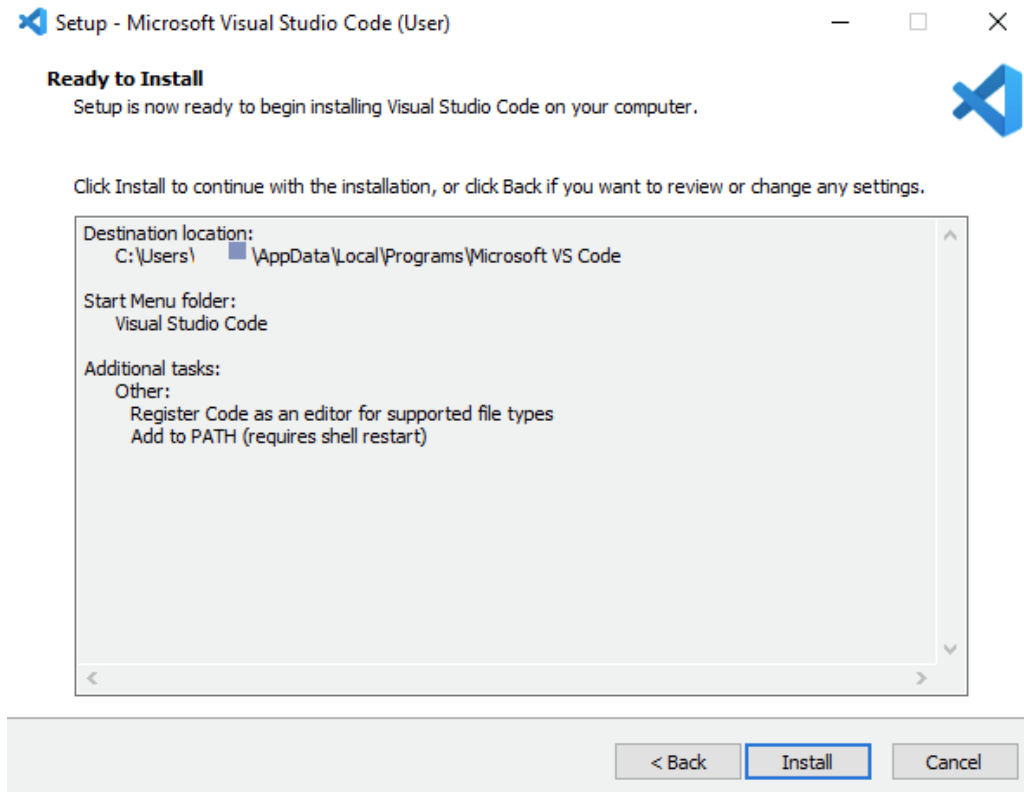
Accept the default Start Menu folder name by clicking Next >.



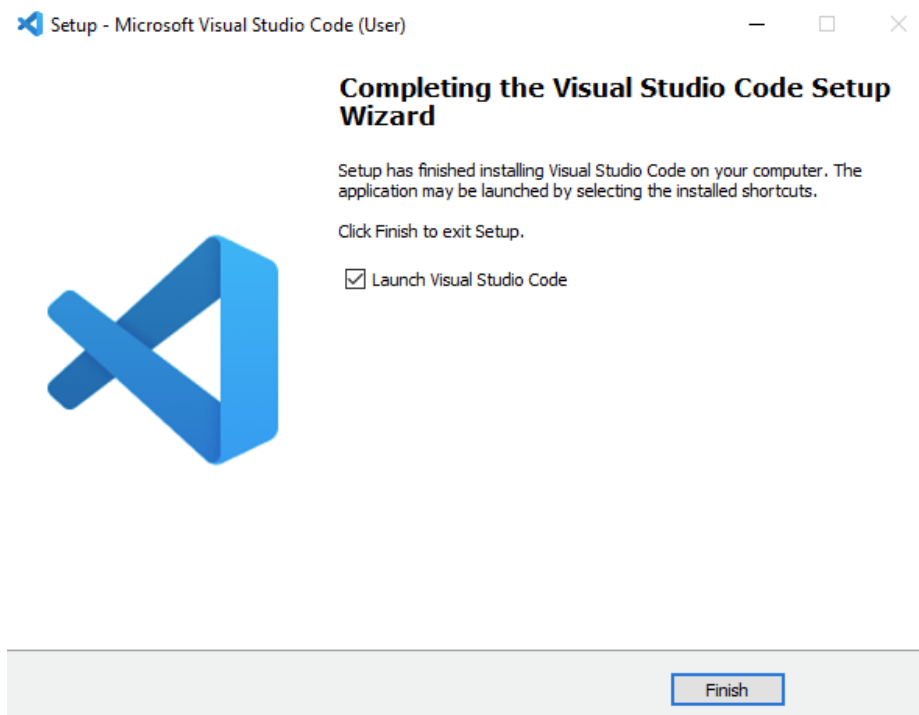
Optionally check the boxes for 'Creating a desktop icon', and adding VS Code to the Right-Click menu functionality of Windows File Explorer, then click Next >.



Confirm the installation options, then click Install.

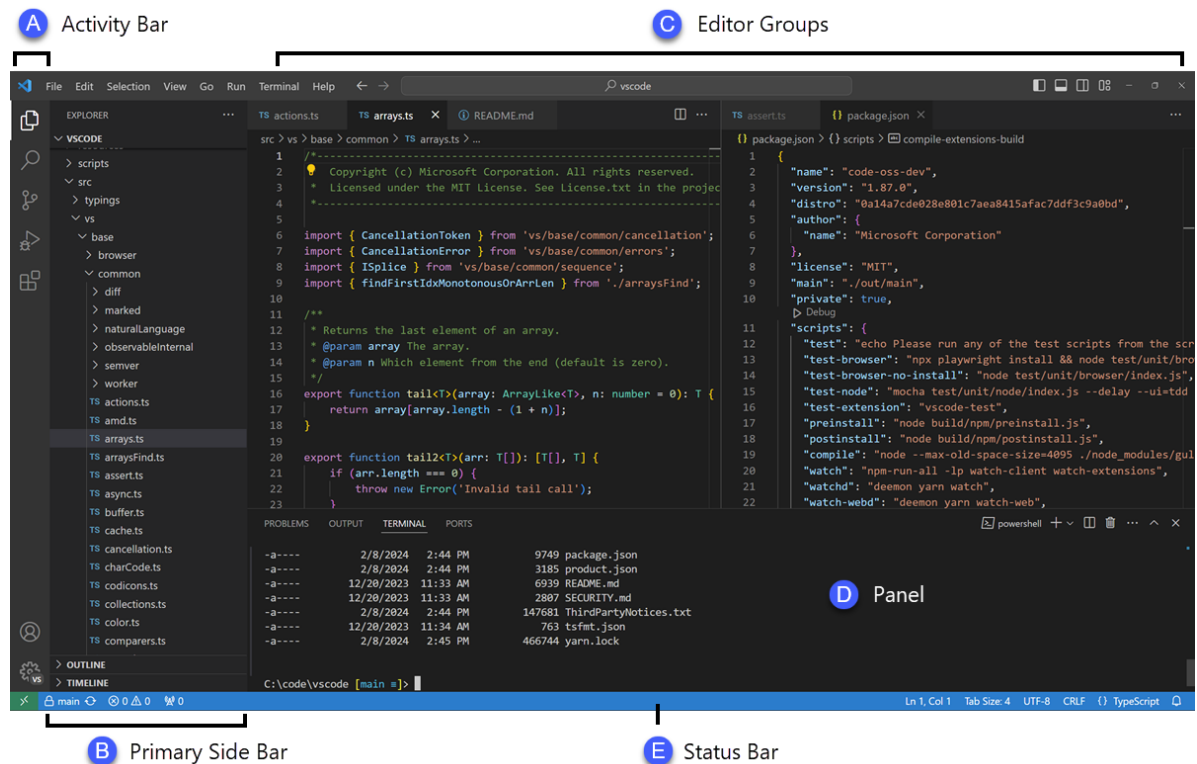


The installation will proceed.



Click Finish to exit the installation and (by default) launch Visual Studio Code:

The Visual Studio Code installer will create an icon in the Start Menu. To locate it, click on the Start Menu and search for 'Code'.



VS Code comes with a simple and intuitive layout that maximizes the space provided for the editor, while leaving ample room to browse and access the full context of your folder or project. The user interface is divided into six main areas:

- **Editor** - The main area to edit your files. You can open as many editors as you like side by side vertically and horizontally.
- **Primary Side Bar** - Contains different views like the Explorer to assist you while working on your project.
- **Secondary Side Bar** - Opposite the Primary Side Bar. By default, contains the Chat view. Drag and drop views from the Primary Side Bar to the Secondary Side Bar to move them.
- **Status Bar** - Information about the opened project and the files you edit.
- **Activity Bar** - Located on the far left-hand side. Lets you switch between views and gives you additional context-specific indicators, like the number of outgoing changes when Git is enabled. You can change the position of the Activity Bar.
- **Panel** - An additional space for views below the editor region. By default, it contains output, debug information, errors and warnings, and an integrated terminal. The Panel can also be moved to the left or right for more vertical space.

Our first small project is to write a program that takes an integer from the user input and tells whether it is a prime number or not. To create a Python program file within VS Code, choose “New File” from File, and then save the file as xyz .py, where xyz is your preferred name for the small project. Here, we use “primetest .py.” VS Code will ask you to choose a folder for the program, and you may create one and then select it to open after the file is created.

The screenshot shows a Windows IDE with a file explorer on the left. The main editor displays a Python script named 'primetest.py'. The script's content is as follows:

```

1  """
2  This program take an integer from user input
3  """
4
5  m = int(input("give me an integer that is greater than 2: "))
6  flag = True
7  if m<2:
8      print("the number must be greater than 2")
9      flag = False

```

Below the editor is a terminal window titled 'TERMINAL'. It shows the command prompt at 'PS D:\workshop\research\books\COMP218\samples>'. The taskbar at the bottom indicates the system is running Python 3.11.1 64-bit.

At this time, you are not required to fully understand the program, but if you are interested, you may type the code line by line into your VS Code and run it by clicking the play button at the top- right corner of the VS Code window, just to get a taste of programming in VS Code with Python. Please note that if you have multiple Python program files open in VS Code and want to run a particular one within the IDE, you will need to click the file to make it active; then, within the editing area, click the right button of the mouse to pop up a menu and select run for your particular program. This is even more important if you have multiple editing tabs open for different programs.

3. Basics of Python

3.1. Basic Output

The `print()` function displays output to the user. Output is the information or result produced by a program. The `sep` and `end` options can be used to customize the output. Table 1 shows examples of `sep` and `end`. Multiple values, separated by commas, can be printed in the same statement.

Example 1: uses of print()

Code	Output
<code>print("Today is Monday.")</code> <code>print("I like string beans.")</code>	Today is Monday. I like string beans.
<code>print("Today", "is", "Monday")</code> <code>print("Today", "is", "Monday", sep="...")</code>	Today is Monday Today...is...Monday
<code>print("Today is Monday, ", end="")</code> <code>print("I like string beans.")</code>	Today is Monday, I like string beans.
<code>print("Today", "is", "Monday", sep="? ", end="!!")</code> <code>print("I like string beans.")</code>	Today? is? Monday!!I like string beans.

By default, each value is separated by a space character in the output. The sep option can be used to change this behavior. By default, the print() function adds a newline character at the end of the output. A newline character tells

3.2. Basic input

Computer programs often receive input from the user. Input is what a user enters into a program. An input statement, `variable = input("prompt")`, has three parts:

1. A variable refers to a value stored in memory. In the statement above, variable can be replaced with any name the programmer chooses.
2. The input() function reads one line of input from the user. A function is a named, reusable block of code that performs a task when called. The input is stored in the computer's memory and can be accessed later using the variable.
3. A prompt is a short message that indicates the program is waiting for input. In the statement above, "prompt" can be omitted or replaced with any message.

Example 2: uses of input()

Code	Output
<code>variable = input("prompt: ")</code> <code>print("the variable = ", variable)</code>	prompt: 5 the variable = 5

3.3. Variables

In Python, variables are names that can be assigned a value and then used to refer to that value throughout your code. Values are assigned to variable names using a special symbol called the assignment operator (=). The = operator takes the value to the right of the operator and assigns it to the name on the left. Ex: `age = 6` or `birth = "May 15"`. The left side of the assignment statement is a variable, and the right side is the value the variable is assigned.

Python has reserved words, known as keywords, which have special functions and cannot be used as names for variables (or other objects).

Table 1: Keywords

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	elif	asynch	del	global
with	yield	if	not	or



Middle Technical University
Technical Engineering College of Artificial Intelligence
Department of Cybersecurity Technology Engineering
Third Year (2025- 2026)

PYTHON PROGRAMMING

Lecture 2: Python Basics

By: Ass. Lec. Abbas Aqeel Kareem

Python Basics

In this section, we will learn the core concepts of Python programming, including variables, data types, operators, control statements, and basic data structures, to build simple and logical Python programs.

Learning Objectives

After completing this chapter, you should be able to

- Understand Python variables, data types, and operators.
- Apply conditional statements and loops to control program flow.
- Manipulate strings and core data structures (lists, tuples, sets, dictionaries, and arrays).
- Develop simple Python programs demonstrating logical and structured problem-solving.

Python Variables

In Python, variables are used to store data that can be referenced and manipulated during program execution. A variable is essentially a name that is assigned to a value. Unlike many other programming languages, Python variables do not require explicit declaration of type. The type of the variable is inferred based on the value assigned.

Variables act as placeholders for data. They allow us to store and reuse values in our program.

Code	Output
<pre># Variable 'age' stores the integer value 29 age = 29 # Variable 'name' stores the string "abbas aqeel" name = "abbas aqeel" print(age) print(name)</pre>	<pre>29 abbas aqeel</pre>

1.1. Rules for Naming Variables

To use variables effectively, we must follow Python's naming rules:

- Variable names can only contain letters, digits and underscores (_).
- A variable name cannot start with a digit.
- Variable names are case-sensitive (myVar and myvar are different).
- Avoid using Python keywords (e.g., if, else, for) as variable names.

Valid	Invalid
<pre>age = 21 _colour = "lilac" total_score = 90</pre>	<pre>1name = "Error" # Starts with a digit class = 10 # 'class' is a reserved keyword user-name = "Doe" # Contains a hyphen</pre>

1.2. Assigning Values to Variables

Variables in Python are assigned values using the = operator

Code
<pre>x = 5 y = 3.14 z = "Hi"</pre>

Python variables are dynamically typed, meaning the same variable can hold different types of values during execution.

Code
<pre>x = 10 x = "Now a string"</pre>

We can assign different values to multiple variables simultaneously, making the code concise and easier to read.

Code	Output
<pre>x, y, z = 1, 2.5, "Python" print(x, y, z)</pre>	1 2.5 Python

1.3. Type Casting a Variable

Type casting refers to the process of converting the value of one data type into another. Python provides several built-in functions to facilitate casting, including:

- `int()`: Converts compatible values to an integer
- `float()`: Transforms values into floating-point numbers.
- `str()`: Converts any data type into a string.

Code	Output
<pre># Casting variables s = "10" # Initially a string n = int(s) # Cast string to integer cnt = 5 f = float(cnt) # Cast integer to float age = 25 s2 = str(age) # Cast integer to string # Display results print(n) print(f) print(s2)</pre>	<pre>10 5.0 25</pre>

1.4. Getting the Type of Variable

In Python, we can determine the type of a variable using the `type()` function. This built-in function returns the type of the object passed to it.

Code	Output
<pre># Define variables with different data types n = 42 f = 3.14 s = "Hello, World!" li = [1, 2, 3] d = {'key': 'value'} bool = True # Get and print the type of each variable print(type(n)) print(type(f)) print(type(s)) print(type(li)) print(type(d)) print(type(bool))</pre>	<pre><class 'int'> <class 'float'> <class 'str'> <class 'list'> <class 'dict'> <class 'bool'></pre>

1.5. Delete a Variable Using `del` Keyword

We can remove a variable from the namespace using the `del` keyword. This effectively deletes the variable and frees up the memory it was using.

Code	Output
<pre># Assigning value to variable x = 10 print(x) # Removing the variable using del del x print(x)</pre>	<pre>10 Traceback (most recent call last): print(x) ^ NameError: name 'x' is not defined</pre>

2. Python Operators

In Python programming, Operators in general are used to perform operations on values and variables. These are standard symbols used for logical and arithmetic operations. In this section, as shown in Table 1 we will look into different types of Python operators.

Table 1: Types of Operators in Python.

Operators	Type
<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>	Arithmetic Operators
<code>==</code> , <code>!=</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code>	Comparison Operators
<code>AND</code> , <code>OR</code> , <code>NOT</code>	Logical Operators
<code>&</code> , <code> </code> , <code>>></code> , <code><<</code> , <code>^</code> , <code>~</code>	Bitwise Operators
<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code>	Assignment Operators

2.1. Arithmetic Operators

Python Arithmetic operators are used to perform basic mathematical operations like addition, subtraction, multiplication and division.

In Python 3.x the result of division is a floating-point while in Python 2.x division of 2 integers was an integer. To obtain an integer result in Python 3.x floored (`//` integer) is used.

Code	Output
<pre># Variables a = 15 b = 4 # Addition print("Addition:", a + b) # Subtraction print("Subtraction:", a - b) # Multiplication print("Multiplication:", a * b) # Division print("Division:", a / b) # Floor Division print("Floor Division:", a // b) # Modulus print("Modulus:", a % b) # Exponentiation print("Exponentiation:", a ** b)</pre>	<pre>Addition: 19 Subtraction: 11 Multiplication: 60 Division: 3.75 Floor Division: 3 Modulus: 3 Exponentiation: 50625</pre>

2.2. Comparison Operators

In Python, Comparison (or Relational) operators compares values. It either returns True or False according to the condition.

Code	Output
<pre>a = 13 b = 33 # greater than print(a > b) # less than print(a < b) # equal to print(a == b) # not equal to print(a != b) # greater or equal print(a >= b) #less or equal print(a <= b)</pre>	<pre>False True False True False True</pre>

2.3. Logical Operators

Python logical operators are used to combine conditional statements, allowing you to perform operations based on multiple conditions. These Python operators, alongside arithmetic operators, are special symbols used to carry out computations on values and variables. In Python, Logical operators are used on conditional statements (either True or False). They perform Logical AND, Logical OR, and Logical NOT operations as shown in table 2.

Table 2: Python Logical Operators

Operator	Description	Syntax
and	Returns True if both the operands are true.	x and y
or	Returns True if either of the operands is true.	x or y
not	Returns True if the operand is false.	not x

Code	Output
<pre># Example: Logical Operators (AND, OR, NOT) with generic variables a, b, c = True, False, True # AND: Both conditions must be True if a and c: print("Both a and c are True (AND condition).") # OR: At least one condition must be True if b or c: print("Either b or c is True (OR condition).") # NOT: Reverses the condition if not b: print("b is False (NOT condition).")</pre>	Both a and c are True (AND condition). Either b or c is True (OR condition). b is False (NOT condition).

2.4. Bitwise Operators

Python bitwise operators as shown in table 3 are used to perform bitwise calculations on integers. The integers are first converted into binary and then operations are performed on each bit or corresponding pair of bits, hence the name bitwise operators. The result is then returned in decimal format.

Table 3: Python bitwise operators.

Operator	Description	Syntax
&	Bitwise AND.	$x \& y$
	Bitwise OR	$x y$
~	Bitwise NOT	$\sim x$
^	Bitwise XOR	$x \wedge y$
>>	Bitwise right shift	$x >>$
<<	Bitwise left shift	$x <<$

Python **Bitwise AND (&)** operator takes two equal-length bit patterns as parameters. The two-bit integers are compared. If the bits in the compared positions of the bit patterns are 1, then the resulting bit is 1. If not, it is 0. For example, Take two bit values X and Y, where $X = 7 = (111)_2$ and $Y = 4 = (100)_2$.

$$\begin{array}{r} 111 \\ \& 100 \\ \hline 100 = 4 \end{array}$$

The Python **Bitwise OR (|)** Operator takes two equivalent length bit designs as boundaries; if the two bits in the looked-at position are 0, the next bit is zero. If not, it is 1. For example, Take two bit values X and Y, where $X = 7 = (111)_2$ and $Y = 4 = (100)_2$. Take Bitwise OR of both X, Y.

$$\begin{array}{r} 111 \\ | 100 \\ \hline 111 = 7 \end{array}$$

The Python **Bitwise XOR (^)** Operator also known as the exclusive OR operator, is used to perform the XOR operation on two operands. XOR stands for "exclusive or", and it returns true if and only if exactly one of the operands is true. In the context of bitwise operations, it compares corresponding bits of two operands. If the bits are different, it returns 1; otherwise, it returns 0. For example, Take two bit values X and Y, where $X = 7 = (111)_2$ and $Y = 4 = (100)_2$. Take Bitwise and of both X & Y.

$$\begin{array}{r} 111 \\ \wedge 100 \\ \hline 011 = 3 \end{array}$$

Python **Bitwise Not (~)** Operator works with a single value and returns its one's complement. This means it toggles all bits in the value, transforming 0 bits to 1 and 1 bits to 0, resulting in the one's complement of the binary number. For example, where $x = 7 = (101)_2$. Python integers are signed and use two's complement arithmetic, so $\sim x = -(x+1) = -(7+1) = -8$.

Bitwise Right Shift (>>) Operator Shifts the bits of the number to the right and fills 0 on voids left (fills 1 in the case of a negative number) as a result. Similar effect as of dividing the number with some power of two.

$$\begin{array}{r} 2 \gg \quad 0000 \ 0111 \\ \hline 0000 \ 0001 = 1 \end{array}$$

Bitwise Left Shift (<<) Operator Shifts the bits of the number to the left and fills 0 on voids right as a result. Similar effect as of multiplying the number with some power of two.

$$\begin{array}{r} 2 \ll \quad 0000 \ 0111 \\ \hline 0001 \ 1100 = 28 \end{array}$$

Code	Output
<code>a = 7</code>	4
<code>b = 4</code>	7
<code>print(a & b)</code>	-8
<code>print(a b)</code>	3
<code>print(~a)</code>	1
<code>print(a ^ b)</code>	28
<code>print(a >> 2)</code>	
<code>print(a << 2)</code>	

2.5. Assignment Operators

Python Assignment operators are used to assign values to the variables. This operator is used to assign the value of the right side of the expression to the left side operand.

The **Addition Assignment** Operator is used to add the right-hand side operand with the left-hand side operand and then assigning the result to the left operand.

The **Subtraction Assignment** Operator is used to subtract the right-hand side operand from the left-hand side operand and then assigning the result to the left-hand side operand.

The **Multiplication Assignment** Operator is used to multiply the right-hand side operand with the left-hand side operand and then assigning the result to the left-hand side operand.

The **Division Assignment** Operator is used to divide the left-hand side operand with the right-hand side operand and then assigning the result to the left operand.

The **Modulus Assignment** Operator is used to take the modulus, that is, it first divides the operands and then takes the remainder and assigns it to the left operand.

Code	Output
<pre> a = 3 b = 5 # a = a + b a += b # Output print("After Addition Assignment a= ", a) # a = a - b a -= b # Output print("After Subtraction Assignment a= ", a) # a = a * b a *= b # Output print("After Multiplication Assignment a= ", a) # a = a / b a /= b # Output print("After Division Assignment a= ", a) # a = a % b a %= b # Output print("After Modulus Assignment a=", a) </pre>	After Addition Assignment a= 8 After Subtraction Assignment a= 3 After Multiplication Assignment a= 15 After Division Assignment a= 3.0 After Modulus Assignment a= 3.0

3. Python Data Types

Data types in Python are a way to classify data items. They represent the kind of value, which determines what operations can be performed on that data. Since everything is an object in Python programming, Python data types are classes and variables are instances (objects) of these classes. The following are standard or built-in data types in Python as shown in figure 1:

- Numeric: Integer, Float, Complex number
- Sequence Type: String, List, Tuple
- Set
- Dictionary
- Boolean
- Binary

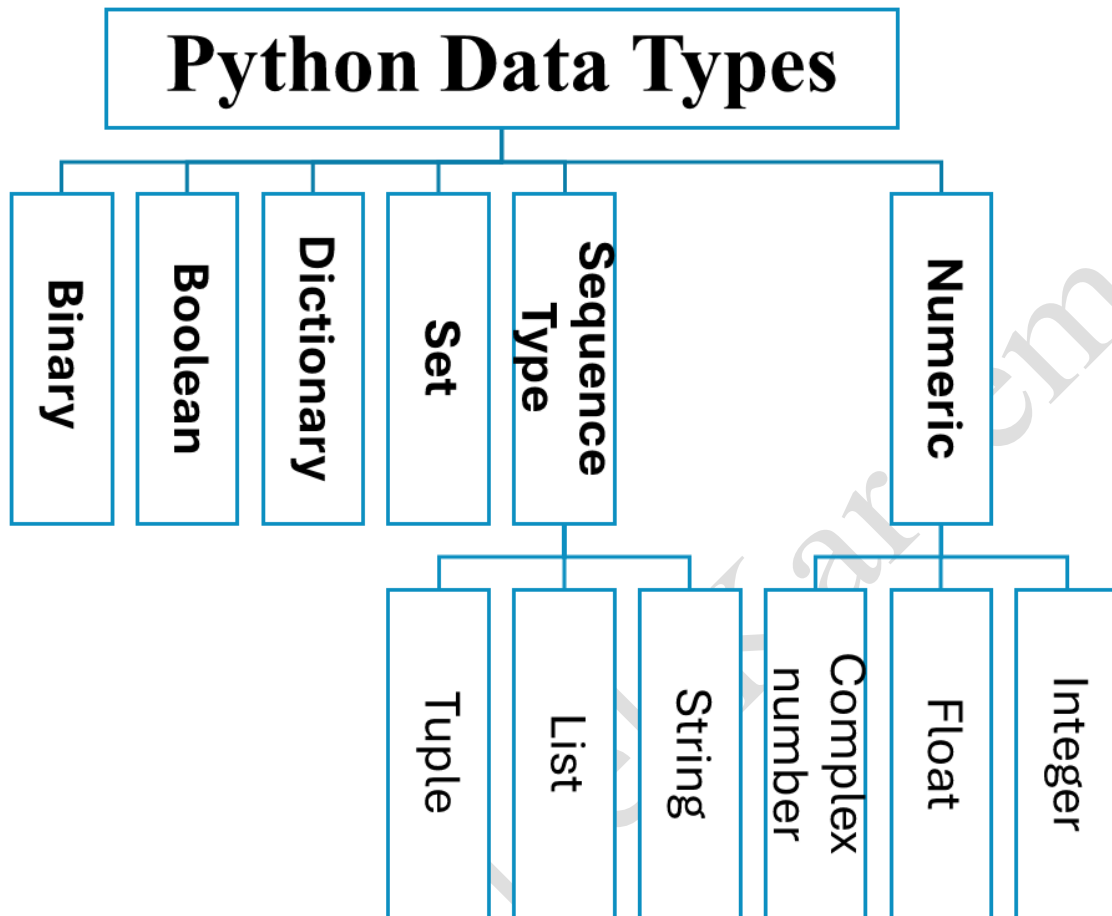


Figure 1: Python Data Types

3.1. Numeric Data Types

Python numbers represent data that has a numeric value. A numeric value can be an integer, a floating number or even a complex number. These values are defined as int, float and complex classes.

- **Integers:** value is represented by int class. It contains positive or negative whole numbers (without fractions or decimals). There is no limit to how long an integer value can be.
- **Float:** value is represented by float class. It is a real number with a floating-point representation. It is specified by a decimal point. Optionally, character e or E followed by a positive or negative integer may be appended to specify scientific notation.
- **Complex Numbers:** It is represented by a complex class. It is specified as (real part) + (imaginary part)j. For example - $2+3j$.

Code	Output
<pre>a = 5 print(type(a)) b = 5.0 print(type(b)) c = 2 + 4j print(type(c))</pre>	<pre><class 'int'> <class 'float'> <class 'complex'></pre>

3.2. Sequence Data Types

A sequence is an ordered collection of items, which can be of similar or different data types. Sequences allow storing of multiple values in an organized and efficient fashion. There are several sequence data types of Python:

- **String Data Type** : Python Strings are arrays of bytes representing Unicode characters. In Python, there is no character data type, a character is a string of length one. It is represented by str class. Strings in Python can be created using single quotes, double quotes or even triple quotes. We can access individual characters of a String using index.

Code	Output
<pre>s = 'Welcome to the python World' print(s) # check data type print(type(s)) # access string with index print(s[1]) print(s[2]) print(s[-1])</pre>	<pre>Welcome to the python World <class 'str'> e l d</pre>

- **List Data Type**: Lists are similar to arrays found in other languages. They are an ordered and mutable collection of items. It is very flexible as items in a list do not need to be of the same type. Lists in Python can be created by just placing sequence inside the square brackets[].

Code	Output
<pre># Empty list a = [] # list with int values a = [1, 2, 3] print(a) # list with mixed values int and String b = ["abbas", "aqeel", "kareem", 4, 5] print(b)</pre>	<pre>[1, 2, 3] ['abbas', 'aqeel', 'kareem', 4, 5]</pre>

- **Tuple Data Type**: Tuple is an ordered collection of Python objects. The only difference between a tuple and a list is that tuples are immutable. Tuples cannot be modified after it is created.

Code	Output
<pre># initiate empty tuple tup1 = () tup2 = ('abbas', 'aqeel') print("\nTuple with the use of String: ", tup2)</pre>	Tuple with the use of String: ('abbas', 'aqeel')

3.3. Boolean Data Type

Python Boolean Data type is one of the two built-in values, True or False. Boolean objects that are equal to True are truthy (true) and those equal to False are falsy (false). However non-Boolean objects can be evaluated in a Boolean context as well and determined to be true or false. It is denoted by class bool.

Code	Output
<pre>print(type(True)) print(type(False)) print(type(true))</pre>	<pre><class 'bool'> <class 'bool'> Traceback (most recent call last): File "/basic.py", line 3, in <module> print(type(true)) ^^^^^ NameError: name 'true' is not defined. Did you mean: 'True'?</pre>

3.4. Set Data Type

In Python Data Types, Set is an unordered collection of data types that is iterable, mutable, and has no duplicate elements. The order of elements in a set is undefined though it may consist of various elements. Sets can be created by using the built-in set() function with an iterable object or a sequence by placing the sequence inside curly braces, separated by a 'comma'. The type of elements in a set need not be the same, various mixed-up data type values can also be passed to the set.

Code	Output
<pre># initializing empty set s1 = set() s1 = set("abbasaqeelkareem") print("Set with the use of String: ", s1) s2 = set(["abbas", "aqeel", "kareem"]) print("Set with the use of List: ", s2)</pre>	Set with the use of String: {'r', 's', 'k', 'l', 'b', 'm', 'q', 'e', 'a'} Set with the use of List: {'aqeel', 'abbas', 'kareem'}

3.5. Dictionary Data Type

A dictionary in Python is a collection of data values, used to store data values like a map, unlike other Python Data Types, a Dictionary holds a key: value pair. Key-value is provided in dictionary to make it more optimized. Each key-value pair in a Dictionary is separated by a colon :, whereas each key is separated by a 'comma'.

Values in a dictionary can be of any datatype and can be duplicated, whereas keys can't be repeated and must be immutable. The dictionary can also be created by the built-in function dict().

Code	Output
<pre># initialize empty dictionary d = {} d = {1: 'abbas', 2: 'aqeel', 3: 'kareem'} print(d) # creating dictionary using dict() constructor d1 = dict({1: 'abbas', 2: 'aqeel', 3: 'kareem'}) print(d1)</pre>	<pre>{1: 'abbas', 2: 'aqeel', 3: 'kareem'} {1: 'abbas', 2: 'aqeel', 3: 'kareem'}</pre>

4. Conditional Statements

Conditional statements in Python are used to execute certain blocks of code based on specific conditions. These statements help control the flow of a program, making it behave differently in different situations.

4.1. If Conditional Statement

If statement is the simplest form of a conditional statement. It executes a block of code if the given condition is true.

Code	Output
<pre>age = 20 if age >= 18: print("Eligible to vote.")</pre>	Eligible to vote.

4.2. If else Conditional Statement

If Else allows us to specify a block of code that will execute if the condition(s) associated with an if or elif statement evaluates to False. Else block provides a way to handle all other cases that don't meet the specified conditions.

Code	Output
<pre>age = 10 if age <= 12: print("Travel for free.") else: print("Pay for ticket.")</pre>	Travel for free.

4.3. elif Statement

elif statement in Python stands for "else if." It allows us to check multiple conditions, providing a way to execute different blocks of code based on which condition is true. Using elif statements makes our code more readable and efficient by eliminating the need for multiple nested if statements.

Code	Output
<pre>age = 25 if age <= 12: print("Child.") elif age <= 19: print("Teenager.") elif age <= 35: print("Young adult.") else: print("Adult.")</pre>	Young adult.

4.4. Nested if..else Conditional Statement

Nested if..else means an if-else statement inside another if statement. We can use nested if statements to check conditions within conditions.

Code	Output
<pre>age = 70 is_member = True if age >= 60: if is_member: print("30% senior discount!") else: print("20% senior discount.") else: print("Not eligible for a senior discount.")</pre>	30% senior discount!

4.5. Match-Case Statement

match-case statement is Python's version of a switch-case found in other languages. It allows us to match a variable's value against a set of patterns.

Code	Output
<pre>number = 2 match number: case 1: print("One") case 2 3: print("Two or Three") case _: print("Other number")</pre>	Two or Three

5. Loops

Loops in Python are used to repeat actions efficiently. The main types are For loops (counting through items) and While loops (based on conditions). In this article, we will look at Python loops and understand their working with the help of examples.

5.1. For Loop

For loops is used to iterate over a sequence such as a list, tuple, string or range. It allow to execute a block of code repeatedly, once for each item in the sequence.

Code	Output
<pre>n = 4 for i in range(0, n): print(i)</pre>	<pre>0 1 2 3</pre>

Using else Statement with for Loop: We can also combine else statement with for loop like in while loop. But as there is no condition in for loop based on which the execution will terminate so the else block will be executed immediately after for block finishes execution.

Code	Output
<pre>li = ["abbas", "aqeel", "kareem"] for index in range(len(li)): print(li[index]) else: print("Inside Else Block")</pre>	<pre>abbas aqeel kareem Inside Else Block</pre>

5.2. While Loop

In Python, a while loop is used to execute a block of statements repeatedly until a given condition is satisfied. When the condition becomes false, the line immediately after the loop in the program is executed. All the statements indented by the same number of character spaces after a programming construct are considered to be part of a single block of code. Python uses indentation as its method of grouping statements. In below code, loop runs as long as the condition `cnt < 3` is true. It increments the counter by 1 on each iteration and prints "Hello Geek" three times.

Code	Output
<pre>cnt = 0 while (cnt < 3): cnt = cnt + 1 print("Hello")</pre>	<pre>Hello Hello Hello</pre>

Using else Statement with for Loop : Else clause is only executed when our while condition becomes false. If we break out of the loop or if an exception is raised then it won't be executed.

Code	Output
<pre>cnt = 0 while (cnt < 3): cnt = cnt + 1 print("Hello") else: print("In Else Block")</pre>	<pre>Hello Hello Hello In Else Block</pre>

Infinite While Loop: If we want a block of code to execute infinite number of times then we can use the while loop in Python to do so.

Code
<pre>while (True): print("Hello Geek")</pre>

5.3. Nested Loops

Python programming language allows to use one loop inside another loop which is called nested loop. Following section shows few examples to illustrate the concept.

Code	Output
<pre>for i in range(1, 5): for j in range(i): print(i, end=' ') print()</pre>	<pre>1 2 2 3 3 3 4 4 4 4</pre>

5.4. Loop Control Statements

Loop control statements change execution from their normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed. Python supports the following control statements.

- **Continue Statement:** The continue statement in Python returns the control to the beginning of the loop.

Code	Output
<pre>for letter in 'abbasaqeel': if letter == 'e' or letter == 's': continue print('Current Letter :', letter)</pre>	<pre>Current Letter : a Current Letter : b Current Letter : b Current Letter : a Current Letter : a Current Letter : q Current Letter : l</pre>

- **Break Statement:** The break statement in Python brings control out of the loop.

Code	Output
<pre>for letter in 'abbasaqeel': if letter == 'e' or letter == 's': break print('Current Letter :', letter)</pre>	<pre>Current Letter : s</pre>

- **Pass Statement:** We use pass statement in Python to write empty loops. Pass is also used for empty control statements, functions and classes.

Code	Output
<pre>for letter in 'abbasaqeel': pass print('Last Letter :', letter)</pre>	Last Letter : l

6. String

In Python, a string is a sequence of characters enclosed in quotes. It can include letters, numbers, symbols or spaces. Since Python has no separate character type, even a single character is treated as a string with length one. Strings are widely used for text handling and manipulation.

6.1.Creating a String

Strings can be created using either single ('...') or double ("...") quotes. Both behave the same.

Code	Output
<pre>s1 = 'abbas' # single quote s2 = "abbas" # double quote print(s1) print(s2)</pre>	abbas abbas

6.2.Multi-line Strings

Use triple quotes ("...") or ("...") for strings that span multiple lines. Newlines are preserved.

Code	Output
<pre>s = """I am Learning Python String on EECT""" print(s) s = '''I'm a student''' print(s)</pre>	I am Learning Python String on EECT I'm a student

6.3.Accessing characters in String

Strings are indexed sequences. Positive indices start at 0 from the left; negative indices start at -1 from the right as represented in below image:

0	1	2	3	4
A	B	B	A	S
-5	-4	-3	-2	-1

Code	Output
<pre>s = "abbas" print(s[0]) # first character print(s[4]) # 5th character print(s[-1]) # last character print(s[-5]) # 5th character from end</pre>	<pre>a s s a</pre>

6.4.String Slicing

Slicing is a way to extract a portion of a string by specifying the start and end indexes. The syntax for slicing is string[start:end], where start starting index and end is stopping index (excluded).

Code	Output
<pre>s = "abbasaqeel" print(s[1:4]) # characters from index 1 to 3 print(s[:3]) # from start to index 2 print(s[3:]) # from index 3 to end print(s[::-1]) # reverse string</pre>	<pre>bba abb asaqeel leeqasabba</pre>

6.5.String Iteration

Strings are iterable; you can loop through characters one by one.

Code	Output
<pre>s = "Python" for char in s: print(char)</pre>	<pre>P y t h o n</pre>

6.6.Deleting a String

In Python, it is not possible to delete individual characters from a string since strings are immutable. However, we can delete an entire string variable using the del keyword.

Code
<pre>s = "abbas" del s</pre>

6.7.Updating a String

As strings are immutable, “updates” create new strings using slicing or methods such as replace().

Code	Output
<pre>s = "abbasaqeel" s1 = "a" + s[1:] s2 = s.replace("abbas", "Abbaskareem") print(s1) print(s2)</pre>	<pre>abbasaqeel Abbaskareemaqeel</pre>

6.8.Common String Methods

Python provides various built-in methods to manipulate strings. Below are some of the most useful methods:

1. len():

The len() function returns the total number of characters in a string (including spaces and punctuation).

Code	Output
<pre>s = "abbasaqeelkareem" print(len(s))</pre>	<pre>16</pre>

2. upper () and lower ()

upper() method converts all characters to uppercase whereas, lower() method converts all characters to lowercase.

Code	Output
<pre>s = "Hello World" print(s.upper()) print(s.lower())</pre>	<pre>HELLO WORLD hello world</pre>

3. strip () and replace ()

strip() removes leading and trailing whitespace from the string and replace() replaces all occurrences of a specified substring with another.

Code	Output
<pre>s = " aba " print(s.strip()) s = "Python is fun" print(s.replace("fun", "awesome"))</pre>	<pre>aba Python is awesome</pre>

6.9.Concatenating and Repeating Strings

We can concatenate strings using + operator and repeat them using * operator.

Code	Output
<pre>s1 = "Hello" s2 = "World" print(s1 + " " + s2) s = "Hello " print(s * 3)</pre>	<pre>Hello World Hello Hello Hello</pre>

6.10. Formatting Strings

Python provides several ways to include variables inside strings.

1. Using f-strings

The simplest and most preferred way to format strings is by using f-strings.

Code	Output
<pre>name = "Alice" age = 22 print(f"Name: {name}, Age: {age}")</pre>	<pre>Name: Alice, Age: 22</pre>

2. Using format()

Another way to format strings is by using format() method.

Code	Output
<pre>name = "Alice" age = 22 print(f"Name: {name}, Age: {age}")</pre>	<pre>My name is Alice and I am 22 years old.</pre>

6.11. String Membership Testing

`in` keyword checks if a particular substring is present in a string.

Code	Output
<pre>s = "abbasaqeel" print("abbas" in s) print("kareem" in s)</pre>	<pre>True False</pre>

7. Lists

In Python, a list is a built-in data structure that can hold an ordered collection of items. Unlike arrays in some languages, Python lists are very flexible:

- Can contain duplicate items
- Mutable: items can be modified, replaced, or removed
- Ordered: maintains the order in which items are added
- Index-based: items are accessed using their position (starting from 0)
- Can store mixed data types (integers, strings, booleans, even other lists)

7.1. Creating a List

Lists can be created in several ways, such as using square brackets, the list() constructor or by repeating elements. Let's look at each method one by one with example:

1. Using Square Brackets

We use square brackets [] to create a list directly.

Code	Output
<pre>a = [1, 2, 3, 4, 5] # List of integers b = ['apple', 'banana', 'cherry'] # List of strings c = [1, 'hello', 3.14, True] # Mixed data types print(a) print(b) print(c)</pre>	<pre>[1, 2, 3, 4, 5] ['apple', 'banana', 'cherry'] [1, 'hello', 3.14, True]</pre>

2. Using list() Constructor

We can also create a list by passing an iterable (like a tuple, string or another list) to the list() function.

Code	Output
<pre>a = list((1, 2, 3, 'apple', 4.5)) print(a) b = list("aba") print(b)</pre>	<pre>[1, 2, 3, 'apple', 4.5] ['a', 'b', 'a']</pre>

3. Creating List with Repeated Elements

We can use the multiplication operator * to create a list with repeated items.

Code	Output
<pre>a = [2] * 5 b = [0] * 7 print(a) print(b)</pre>	<pre>[2, 2, 2, 2, 2] [0, 0, 0, 0, 0, 0, 0]</pre>

7.2. Accessing List Elements

Elements in a list are accessed using indexing. Python indexes start at 0, so a[0] gives the first element. Negative indexes allow access from the end (e.g., -1 gives the last element).

Code	Output
<pre>a = [10, 20, 30, 40, 50] print(a[0]) print(a[-1]) print(a[1:4]) # elements from index 1 to 3</pre>	<pre>10 50 [20, 30, 40]</pre>

7.3.Adding Elements into List

We can add elements to a list using the following methods:

- `append()`: Adds an element at the end of the list.
- `extend()`: Adds multiple elements to the end of the list.
- `insert()`: Adds an element at a specific position.
- `clear()`: removes all items.

Code	Output
<pre>a = [] a.append(10) print("After append(10):", a) a.insert(0, 5) print("After insert(0, 5):", a) a.extend([15, 20, 25]) print("After extend([15, 20, 25]):", a) a.clear() print("After clear():", a)</pre>	<pre>After append(10): [10] After insert(0, 5): [5, 10] After extend([15, 20, 25]): [5, 10, 15, 20, 25] After clear(): []</pre>

7.4.Updating Elements into List

Since lists are mutable, we can update elements by accessing them via their index.

Code	Output
<pre>a = [10, 20, 30, 40, 50] a[1] = 25 print(a)</pre>	<pre>[10, 25, 30, 40, 50]</pre>

7.5.Removing Elements from List

We can remove elements from a list using:

- `remove()`: Removes the first occurrence of an element.
- `pop()`: Removes the element at a specific index or the last element if no index is specified.
- `del` statement: Deletes an element at a specified index.

Code	Output
<pre>a = [10, 20, 30, 40, 50] a.remove(30) print("After remove(30):", a) popped_val = a.pop(1) print("Popped element:", popped_val) print("After pop(1):", a) del a[0] print("After del a[0]:", a)</pre>	<pre>After remove(30): [10, 20, 40, 50] Popped element: 20 After pop(1): [10, 40, 50] After del a[0]: [40, 50]</pre>

7.6.Iterating Over Lists

We can iterate over lists using loops, which is useful for performing actions on each item.

Code	Output
<pre>a = ['apple', 'banana', 'cherry'] for item in a: print(item)</pre>	<pre>apple banana cherry</pre>

7.7.Nested Lists

A nested list is a list within another list, which is useful for representing matrices or tables. We can access nested elements by chaining indexes.

Code	Output
<pre>matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] print(matrix[1][2])</pre>	<pre>6</pre>

8. Tuples

A tuple in Python is an immutable ordered collection of elements.

- Tuples are similar to lists, but unlike lists, they cannot be changed after their creation (i.e., they are immutable).
- Tuples can hold elements of different data types.
- The main characteristics of tuples are being ordered, heterogeneous and immutable.

8.1.Creating a Tuple

A tuple is created by placing all the items inside parentheses (), separated by commas. A tuple can have any number of items and they can be of different data types.

Code	Output
<pre> tup = () print(tup) # Using String tup = ('Geeks', 'For') print(tup) # Using List li = [1, 2, 4, 5, 6] print(tuple(li)) # Using Built-in Function tup = tuple('Geeks') print(tup) </pre>	<pre> () ('Geeks', 'For') (1, 2, 4, 5, 6) ('G', 'e', 'e', 'k', 's') </pre>

8.2. Creating a Tuple with Mixed Datatypes

Tuples can contain elements of various data types, including other tuples, lists, dictionaries and even functions.

Code	Output
<pre> tup = (5, 'Welcome', 7, 'python') print(tup) # Creating a Tuple with nested tuples tup1 = (0, 1, 2, 3) tup2 = ('python', 'fun') tup3 = (tup1, tup2) print(tup3) # Creating a Tuple with repetition tup1 = ('python',) * 3 print(tup1) # Creating a Tuple with the use of loop tup = ('python') n = 5 for i in range(int(n)): tup = (tup,) print(tup) </pre>	<pre> (5, 'Welcome', 7, 'python') ((0, 1, 2, 3), ('python', 'fun')) ('python', 'python', 'python') ('python',) (('python',),) ((('python',),),) (((('python',),),),) ((((('python',),),),),) </pre>

8.3. Tuple Basic Operations

1. Accessing of Tuples

We can access the elements of a tuple by using indexing and slicing, similar to how we access elements in a list. Indexing starts at 0 for the first element and goes up to n-1, where n is the number of elements in the tuple. Negative indexing starts from -1 for the last element and goes backward.

Code	Output
<pre># Accessing Tuple with Indexing tup = tuple("python") print(tup[0]) # Accessing a range of elements using slicing print(tup[1:4]) print(tup[:3]) # Tuple unpacking tup = ("python", "For", "students") # This line unpack values of Tuple1 a, b, c = tup print(a) print(b) print(c)</pre>	<pre>p ('y', 't', 'h') ('p', 'y', 't') python For students</pre>

2. Concatenation of Tuples

Tuples can be concatenated using the + operator. This operation combines two or more tuples to create a new tuple.

Code	Output
<pre>tup1 = (0, 1, 2, 3) tup2 = ('Python', 'For', 'Student') tup3 = tup1 + tup2 print(tup3)</pre>	<pre>(0, 1, 2, 3, 'Python', 'For', 'Student')</pre>

3. Slicing of Tuple

Slicing a tuple means creating a new tuple from a subset of elements of the original tuple. The slicing syntax is tuple[start:stop:step].

Code	Output
<pre>tup = tuple('abbasaqeel') # Removing First element print(tup[1:]) # Reversing the Tuple print(tup[::-1]) # Printing elements of a Range print(tup[4:9])</pre>	<pre>('b', 'b', 'a', 's', 'a', 'q', 'e', 'e', 'l') ('l', 'e', 'e', 'q', 'a', 's', 'a', 'b', 'b', 'a') ('s', 'a', 'q', 'e', 'e')</pre>

4. Deleting a Tuple

Since tuples are immutable, we cannot delete individual elements of a tuple. However, we can delete an entire tuple using del statement.

Code
<pre>tup = (0, 1, 2, 3, 4) del tup</pre>

5. Tuple Unpacking with Asterisk (*)

In Python, the "*" operator can be used in tuple unpacking to grab multiple items into a list. This is useful when you want to extract just a few specific elements and collect the rest together.

Code	Output
<pre>tup= (1, 2, 3, 4, 5) a, *b, c = tup print(a) print(b) print(c)</pre>	<pre>1 [2, 3, 4] 5</pre>

9. Dictionary

Python dictionary is a data structure that stores the value in key: value pairs. Values in a dictionary can be of any data type and can be duplicated, whereas keys can't be repeated and must be immutable.

- Keys are case sensitive which means same name but different cases of Key will be treated distinctly.
- Keys must be immutable which means keys can be strings, numbers or tuples but not lists.
- Duplicate keys are not allowed and any duplicate key will overwrite the previous value.
- Internally uses hashing. Hence, operations like search, insert, delete can be performed in Constant Time.
- From Python 3.7 Version onward, Python dictionary are Ordered.

9.1.Create a Dictionary

Dictionary can be created by placing a sequence of elements within curly {} braces, separated by a 'comma'.

Code	Output
<pre>d1 = {1: 'python', 2: 'For', 3: 'cyber'} print(d1) # create dictionary using dict() constructor d2 = dict(a = "python", b = "for", c = "cyber") print(d2)</pre>	<pre>{1: 'python', 2: 'For', 3: 'cyber'} {'a': 'python', 'b': 'for', 'c': 'cyber'}</pre>

9.2.Accessing Dictionary Items

We can access a value from a dictionary by using the key within square brackets or get() method.

Code	Output
<pre>d = { "name": "Prajjwal", 1: "Python", (1, 2): [1,2,4] } # Access using key print(d["name"]) # Access using get() print(d.get("name"))</pre>	<pre>Prajjwal Prajjwal</pre>

9.3.Adding and Updating Dictionary Items

We can add new key-value pairs or update existing keys by using assignment.

Code	Output
<pre>d = {1: 'python', 2: 'For', 3: 'cyber'} # Adding a new key-value pair d["age"] = 22 # Updating an existing value d[1] = "Python dict" print(d)</pre>	<pre>{1: 'Python dict', 2: 'For', 3: 'cyber', 'age': 22}</pre>

9.4.Removing Dictionary Items

We can remove items from dictionary using the following methods:

- del: Removes an item by key.
- pop(): Removes an item by key and returns its value.
- clear(): Empties the dictionary.
- popitem(): Removes and returns the last key-value pair.

Code	Output
<pre>d = {1: 'python', 2: 'For', 3: 'student', 'age':22} # Using del to remove an item del d["age"] print(d) # Using pop() to remove an item and return the value val = d.pop(1) print(val) # Using popitem to removes and returns # the last key-value pair. key, val = d.popitem() print(f"Key: {key}, Value: {val}") # Clear all items from the dictionary d.clear() print(d)</pre>	<pre>{1: 'python', 2: 'For', 3: 'student'} python Key: 3, Value: student {} </pre>

9.5. Iterating Through a Dictionary

We can iterate over keys [using keys() method] , values [using values() method] or both [using item() method] with a for loop.

Code	Output
<pre> d = {1: 'python', 2: 'For', 3: 'student', 'age':22} # Using del to remove an item del d["age"] print(d) # Using pop() to remove an item and return the value val = d.pop(1) print(val) # Using popitem to removes and returns # the last key-value pair. key, val = d.popitem() print(f"Key: {key}, Value: {val}") # Clear all items from the dictionary d.clear() print(d) </pre>	<pre> {1: 'python', 2: 'For', 3: 'student'} python Key: 3, Value: student {} </pre>

10. Sets

Python set is an unordered collection of multiple items having different datatypes. In Python, sets are mutable, unindexed and do not contain duplicates. The order of elements in a set is not preserved and can change.

- Can store None values.
- Implemented using hash tables internally.
- Do not implement interfaces like Serializable or Cloneable.
- Python sets are not inherently thread-safe; synchronization is needed if used across threads.

10.1. Creating a Set

In Python, the most basic and efficient method for creating a set is using curly braces. Also Python Sets can be created by using the built-in set() function with an iterable object or a sequence by placing the sequence inside curly braces, separated by a 'comma'.

Code	Output
<pre> set1 = {1, 2, 3, 4} print(set1) set1 = set() print(set1) set1 = set("PythonforCyber") print(set1) # Creating a Set with the use of a List set1 = set(["Python", "For", "Cyber"]) print(set1) # Creating a Set with the use of a tuple tup = ("Python", "for", "Cyber") print(set(tup)) # Creating a Set with the use of a dictionary d = {"Python": 1, "for": 2, "Cyber": 3} print(set(d)) </pre>	<pre> {1, 2, 3, 4} set() {'t', 'n', 'f', 'y', 'r', 'e', 'P', 'h', 'b', 'o', 'C'} {'Python', 'For', 'Cyber'} {'Python', 'for', 'Cyber'} {'Python', 'for', 'Cyber'} </pre>

10.2. Adding Elements to a Set

We can add items to a set using add() method and update() method. add() method can be used to add only a single item. To add multiple items we use update() method.

Code	Output
<pre> # Creating a set set1 = {1, 2, 3} # Add one item set1.add(4) # Add multiple items set1.update([5, 6]) print(set1) </pre>	<pre> {1, 2, 3, 4, 5, 6} </pre>

10.3. Accessing a Set

We can loop through a set to access set items as set is unindexed and do not support accessing elements by indexing. Also we can use in keyword which is membership operator to check if an item exists in a set.

Code	Output
<pre> set1 = set(["Python", "For", "Cyber."]) # Accessing element using For loop for i in set1: print(i, end=" ") # Checking the element# using in keyword print("Python" in set1) </pre>	<pre> Python For Cyber. True </pre>

10.4. Removing Elements from the Set

We can remove an element from a set in Python using several methods: `remove()`, `discard()` and `pop()`. Each method works slightly differently:

- Using **remove() Method or discard() Method**: `remove()` method removes a specified element from the set. If the element is not present in the set, it raises a `KeyError`. `discard()` method also removes a specified element from the set. Unlike `remove()`, if the element is not found, it does not raise an error.

Code	Output
<pre># Using Remove Method set1 = {1, 2, 3, 4, 5} set1.remove(3) print(set1) # Attempting to remove an element that does not exist try: set1.remove(10) except KeyError as e: print("Error:", e) # Using discard() Method set1.discard(4) print(set1) # Attempting to discard an element that does not exist set1.discard(10) # No error raised print(set1)</pre>	<pre>{1, 2, 4, 5} Error: 10 {1, 2, 5} {1, 2, 5}</pre>

- Using **pop() Method**: `pop()` method removes and returns an arbitrary element from the set. This means we don't know which element will be removed. If the set is empty, it raises a `KeyError`.

Code	Output
<pre>set1 = {1, 2, 3, 4, 5} val = set1.pop() print(val) print(set1) # Using pop on an empty set set1.clear() # Clear the set to make it empty try: set1.pop() except KeyError as e: print("Error:", e)</pre>	<pre>1 {2, 3, 4, 5} Error: 'pop from an empty set'</pre>

- Using **clear() Method**: `clear()` method removes all elements from the set, leaving it empty.

Code	Output
<pre>set1 = {1, 2, 3, 4, 5} set1.clear() print(set1)</pre>	<pre>set()</pre>

10.5. Frozen Sets

A frozenset in Python is a built-in data type that is similar to a set but with one key difference that is immutability. This means that once a frozenset is created, we cannot modify its elements that is we cannot add, remove or change any items in it. Like regular sets, a frozenset cannot contain duplicate elements.

Code	Output
<pre># Creating a frozenset from a list fset = frozenset([1, 2, 3, 4, 5]) print(fset) # Creating a frozenset from a set set1 = {3, 1, 4, 1, 5} fset = frozenset(set1) print(fset)</pre>	<pre>frozenset({1, 2, 3, 4, 5}) frozenset({1, 3, 4, 5})</pre>

11. Arrays

In Python, array is a collection of items stored at contiguous memory locations. The idea is to store multiple items of the same type together. Unlike Python lists (can store elements of mixed types), arrays must have all elements of same type. Having only homogeneous elements makes it memory-efficient.

Code	Output
<pre>import array as arr a = arr.array('i', [1, 2, 3]) # accessing First Array print(a[0]) # adding element to array a.append(5) print(a)</pre>	<pre>1 array('i', [1, 2, 3, 5])</pre>

11.1. Create an Array

Array in Python can be created by importing an array module. `array( data_type , value_list )` is used to create array in Python with data type and value list specified in its arguments.

Code	Output
<pre>import array as arr a = arr.array('i', [1, 2, 3]) for i in range(0, 3): print(a[i], end=" ")</pre>	<pre>1 2 3</pre>

11.2. Adding Elements to an Array

Elements can be added to the Python Array by using built-in insert() function. Insert is used to insert one or more data elements into an array. Based on the requirement, a new element can be added at the beginning, end, or any given index of array. append() is also used to add the value mentioned in its arguments at the end of the Python array.

Code	Output
<pre>import array as arr a = arr.array('i', [1, 2, 3]) print(*a) a.insert(1, 4) # Insert 4 at index 1 print(*a)</pre>	<pre>1 2 3 1 4 2 3</pre>

11.3. Accessing Array Items

In order to access the array items refer to the index number. Use the index operator [] to access an item in a array in Python. The index must be an integer.

Code	Output
<pre>import array as arr a = arr.array('i', [1, 2, 3, 4, 5, 6]) print(a[0]) print(a[3]) b = arr.array('d', [2.5, 3.2, 3.3]) print(b[1]) print(b[2])</pre>	<pre>1 4 3.2 3.3</pre>

11.4. Removing Elements from the Array

Elements can be removed from the Python array by using built-in remove() function. It will raise an Error if element doesn't exist. Remove() method only removes the first occurrence of the searched element. To remove range of elements, we can use an iterator. pop() function can also be used to remove and return an element from the array. By default it removes only the last element of the array. To remove element from a specific position, index of that item is passed as an argument to pop() method.

Code	Output
<pre>import array a = array.array('i', [1, 2, 3, 1, 5]) # remove first occurrence of 1 a.remove(1) print(a) # remove item at index 2 a.pop(2) print(a)</pre>	<pre>array('i', [2, 3, 1, 5]) array('i', [2, 3, 5])</pre>

11.5. Slicing of an Array

In Python array, there are multiple ways to print the whole array with all the elements, but to print a specific range of elements from the array, we use Slice operation:

- Elements from beginning to a range use [:Index]
- Elements from end use[:-Index]
- Elements from specific Index till the end use [Index:]
- Elements within a range, use [Start Index:End Index]
- Print complete List, use [:].
- For Reverse list, use[::-1].

Code	Output
<pre>import array as arr a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] b = arr.array('i', a) res = a[3:8] print(res) res = a[5:] print(res) res = a[:] print(res)</pre>	<pre>[4, 5, 6, 7, 8] [6, 7, 8, 9, 10] [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]</pre>

11.6. Searching Element in an Array

In order to search an element in the array we use a python in-built index() method. This function returns the index of the first occurrence of value mentioned in arguments.

Code	Output
<pre>import array a = array.array('i', [1, 2, 3, 1, 2, 5]) # index of 1st occurrence of 2 print(a.index(2)) # index of 1st occurrence of 1 print(a.index(1))</pre>	<pre>1 0</pre>

11.7. Updating Elements in an Array

In order to update an element in the array we simply reassign a new value to the desired index we want to update.

Code	Output
<pre>import array a = array.array('i', [1, 2, 3, 1, 2, 5]) # update item at index 2 a[2] = 6 print(a) # update item at index 4 a[4] = 8 print(a)</pre>	<pre>array('i', [1, 2, 6, 1, 2, 5]) array('i', [1, 2, 6, 1, 8, 5])</pre>

11.8. Different Operations on Python Arrays

1. Counting Elements in an Array

We can use count() method to count given item in array.

Code	Output
<pre>import array a = array.array('i', [1, 2, 3, 4, 2, 5, 2]) count = a.count(2) print(count)</pre>	3

2. Reversing Elements in an Array

In order to reverse elements of an array we need to simply use reverse method.

Code	Output
<pre>import array a = array.array('i', [1, 2, 3, 4, 5]) a.reverse() print(*a)</pre>	5 4 3 2 1

3. Extend Element from Array

In Python, an array is used to store multiple values or elements of the same datatype in a single variable. The extend() function is simply used to attach an item from iterable to the end of the array. In simpler terms, this method is used to add an array of values to the end of a given or existing array.

Code	Output
<pre>import array as arr a = arr.array('i', [1, 2, 3, 4, 5]) a.extend([6, 7, 8, 9, 10]) print(a)</pre>	<pre>array('i', [1, 2, 3, 4, 5, 6, 7, 8, 9, 10])</pre>



Middle Technical University

Technical Engineering College of Artificial Intelligence

Department of Cybersecurity Technology Engineering

Third Year

PYTHON PROGRAMMING

Lecture 3: Functions & OOP

By: Ass. Lec. Abbas Aqeel Kareem

Python Functions and OOP

In this section of Python 3 tutorial we'll explore Python function syntax, parameter handling, return values and variable scope. Along the way, we'll also introduce versatile functions like `range()`, `map`, `filter` and `lambda` functions.

Learning Objectives

After completing this chapter, you should be able to

- Discuss computer programming languages.
- Describe Python and discuss its development, features, and advantages.
- Install Python and required Python IDEs on the computer.
- Get Python and Python IDEs running for the learning activities included in this textbook.

1. Functions

Python Functions are a block of statements that does a specific task. The idea is to put some commonly or repeatedly done task together and make a function so that instead of writing the same code again and again for different inputs, we can do the function calls to reuse code contained in it over and over again.

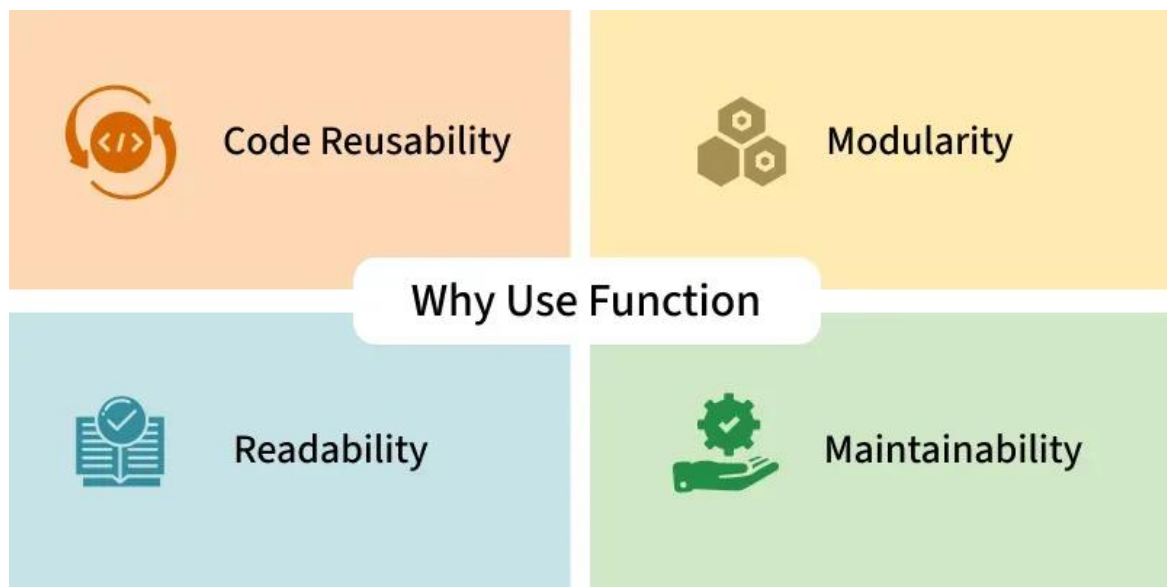


Figure 1: key reasons for using functions in programming

1.1.Function Declaration

We can define a function in Python, using the `def` keyword. A function might take input in the form of parameters. The syntax to declare a function is:

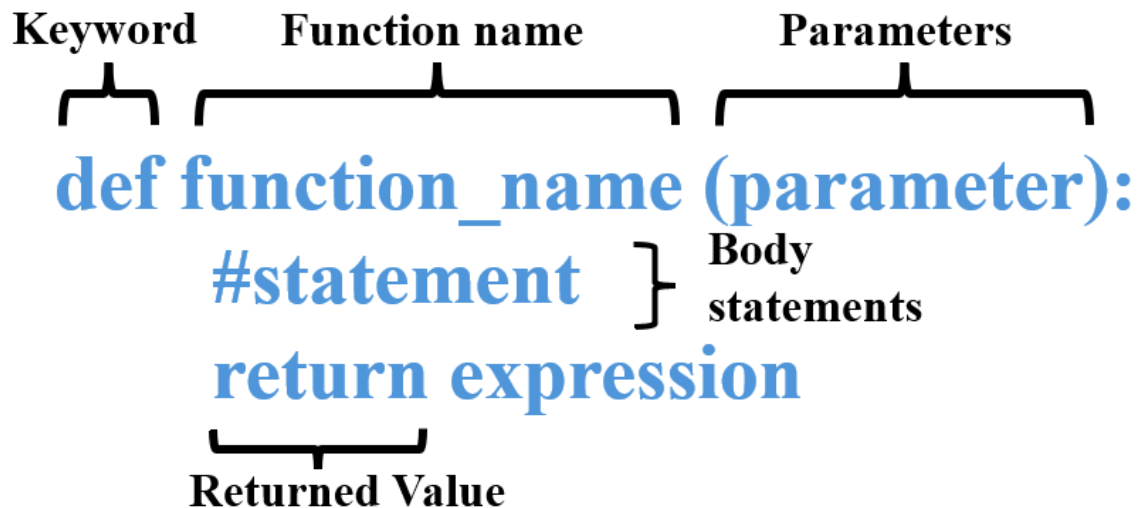


Figure 2: Syntax of Python Function Declaration

1.2. Defining a Function

Here, we define a function using def that prints a welcome message when called.

Code
<pre>def fun(): print("Welcome to Python")</pre>

1.3. Calling a Function

After creating a function in Python we can call it by using the name of the functions followed by parenthesis containing parameters of that particular function.

Code	Output
<pre>def fun(): print("Welcome to Python") fun() # Driver code to call a function</pre>	Welcome to Python

1.4. Function Arguments

Arguments are the values passed inside the parenthesis of the function. A function can have any number of arguments separated by a comma. We will create a simple function in Python to check whether the number passed as an argument to the function is even or odd.

Code	Output
<pre>def evenOdd(x): if (x % 2 == 0): return "Even" else: return "Odd" print(evenOdd(16))</pre>	Even Odd

<code>print(evenOdd(7))</code>	
--------------------------------	--

1.5.Types of Function Arguments

Python supports various types of arguments that can be passed at the time of the function call. In Python, we have the following function argument types in Python, Let's explore them one by one.

1. Default Arguments

A default argument is a parameter that assumes a default value if a value is not provided in the function call for that argument.

Code	Output
<pre>def myFun(x, y=50): print("x: ", x) print("y: ", y) myFun(10)</pre>	<pre>x: 10 y: 50</pre>

2. Keyword Arguments

In keyword arguments, values are passed by explicitly specifying the parameter names, so the order doesn't matter.

Code	Output
<pre>def student(fname, lname): print(fname, lname) student(fname='Python', lname='Practice') student(lname='Practice', fname='Python')</pre>	<pre>Python Practice Python Practice</pre>

3. Positional Arguments

In positional arguments, values are assigned to parameters based on their order in the function call.

Code	Output
<pre>def nameAge(name, age): print("Hi, I am", name) print("My age is ", age) print("Case-1:") nameAge("Suraj", 27) print("\nCase-2:") nameAge(27, "Suraj")</pre>	<pre>Hi, I am Suraj My age is 27 Case-2: Hi, I am 27 My age is Suraj</pre>

4. Arbitrary Arguments

In Python Arbitrary Keyword Arguments, *args and **kwargs can pass a variable number of arguments to a function using special symbols. There are two special symbols:

- ***args** in Python (Non-Keyword Arguments)
- ****kwargs** in Python (Keyword Arguments)

This code separately shows non-keyword (***args**) and keyword (****kwargs**) arguments in the same function.

Code	<pre>def myFun(*args, **kwargs): print("Non-Keyword Arguments (*args):") for arg in args: print(arg) print("\nKeyword Arguments (**kwargs):") for key, value in kwargs.items(): print(f"{key} == {value}") # Function call with both types of arguments myFun('Hey', 'Welcome', first='Python', mid='for', last='Cyber')</pre>
Output	<pre>Non-Keyword Arguments (*args): Hey Welcome Keyword Arguments (**kwargs): first == Python mid == for last == Cyber</pre>

1.6.Function within Functions

A function defined inside another function is called an inner function (or nested function). It can access variables from the enclosing function's scope and is often used to keep logic protected and organized.

Code	Output
<pre>def f1(): s = 'I love Python' def f2(): print(s) f2() f1()</pre>	I love Python

1.7. Anonymous Functions

In Python, an anonymous function means that a function is without a name. As we already know the `def` keyword is used to define the normal functions and the `lambda` keyword is used to create anonymous functions.

Code	Output
<pre>def cube(x): return x*x*x # without lambda cube_1 = lambda x : x*x*x # with lambda</pre>	<pre>343 343</pre>

<pre>print(cube(7)) print(cube_1(7))</pre>	
--	--

1.8.Return Statement in Function

The return statement ends a function and sends a value back to the caller. It can return any data type, multiple values (packed into a tuple), or None if no value is given.

Code	Output
<pre>def square_value(num): """This function returns the square value of the entered number""" return num**2 print(square_value(2)) print(square_value(-4))</pre>	<pre>343 343</pre>

1.9.Recursive Functions

A recursive function is a function that calls itself to solve a problem. It is commonly used in mathematical and divide-and-conquer problems. Always include a base case to avoid infinite recursion.

Code	Output
<pre>def factorial(n): if n == 0: return 1 else: return n * factorial(n - 1) print(factorial(4))</pre>	<pre>24</pre>

2. OOP

Object Oriented Programming is a fundamental concept in Python, empowering developers to build modular, maintainable and scalable applications. OOP is a way of organizing code that uses objects and classes to represent real-world entities and their behavior. In OOP, object has attributes thing that has specific data and can perform certain actions using methods.

Python supports the core principles of object-oriented programming, which are the building blocks for designing robust and reusable software. The figure below demonstrates these core principles:

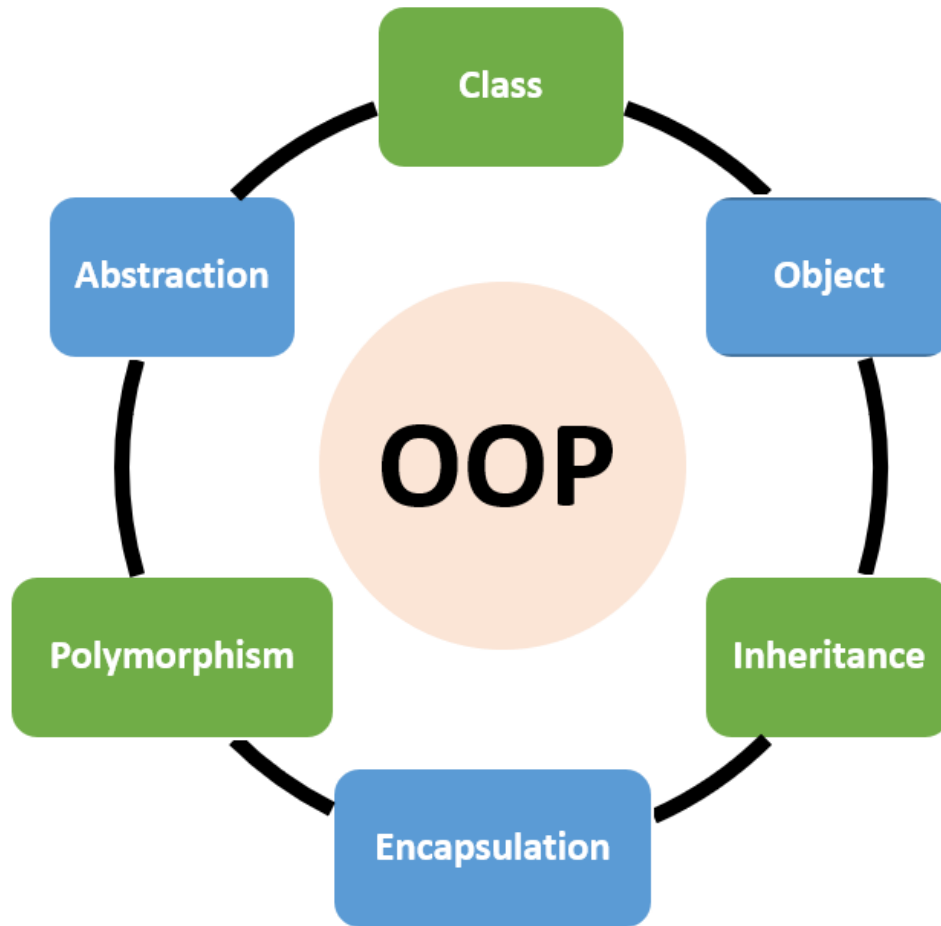


Figure 3: Core Concepts of Object-Oriented Programming (OOP).

2.1.Class

A class is a collection of objects. Classes are blueprints for creating objects. A class defines a set of attributes and methods that the created objects (instances) can have.

Some points on Python class:

1. Classes are created by keyword `class`.
2. Attributes are the variables that belong to a class.
3. Attributes are always public and can be accessed using the dot (`.`) operator.

Creating a Class

Here, `class` keyword indicates that we are creating a class followed by name of the class (Dog in this case).

Code	<pre>class Dog: species = "Canine" # Class attribute def __init__(self, name, age): self.name = name # Instance attribute self.age = age # Instance attribute</pre>
Explanation	<code>class Dog</code>: Defines a class named Dog. <code>species</code>: A class attribute shared by all instances of the class. <code>__init__</code> method: Initializes the name and age attributes when a new object is created.

2.2.Objects

An Object is an instance of a Class. It represents a specific implementation of the class and holds its own data. An object consists of:

1. State: It is represented by the attributes and reflects the properties of an object.
2. Behavior: It is represented by the methods of an object and reflects the response of an object to other objects.
3. Identity: It gives a unique name to an object and enables one object to interact with other objects.

Creating Object

Creating an object in Python involves instantiating a class to create a new instance of that class. This process is also referred to as object instantiation.

Code	<pre>class Dog: species = "Canine" # Class attribute def __init__(self, name, age): self.name = name # Instance attribute self.age = age # Instance attribute # Creating an object of the Dog class dog1 = Dog("Buddy", 3) print(dog1.name) print(dog1.species)</pre>
Output	<pre>Buddy Canine</pre>
Explanation	<code>dog1 = Dog("Buddy", 3)</code>: Creates an object of the Dog class with name as "Buddy" and age as 3. <code>dog1.name</code>: Accesses the instance attribute name of the dog1 object. <code>dog1.species</code>: Accesses the class attribute species of the dog1 object.

Self Parameter

Self parameter is a reference to the current instance of the class. It allows us to access the attributes and methods of the object. In this example, we create a Dog class with both class and instance attributes, then demonstrate how to access them using the self parameter.

Code	<pre> class Dog: species = "Canine" # Class attribute def __init__(self, name, age): self.name = name # Instance attribute self.age = age # Instance attribute dog1 = Dog("Buddy", 3) # Create an instance of Dog dog2 = Dog("Charlie", 5) # Create another instance of Dog print(dog1.name, dog1.age, dog1.species) # Access instance and class attributes print(dog2.name, dog2.age, dog2.species) # Access instance and class attributes print(Dog.species) # Access class attribute directly </pre>
Output	<pre> Buddy 3 Canine Charlie 5 Canine Canine </pre>
Explanation	<code>self.name</code>: Refers to the name attribute of the object (dog1) calling the method. <code>dog1.bark()</code>: Calls the bark method on dog1.

__init__ Method

`__init__` method is the constructor in Python, automatically called when a new object is created. It initializes the attributes of the class. In this example, we create a Dog class and use `__init__` method to set the name and age of each dog when creating an object.

Code	<pre> class Dog: def __init__(self, name, age): self.name = name self.age = age dog1 = Dog("Buddy", 3) print(dog1.name) </pre>
Output	<pre> Buddy </pre>
Explanation	<code>__init__</code>: Special method used for initialization. <code>self.name</code> and <code>self.age</code>: Instance attributes initialized in the constructor.

Class and Instance Variables

In Python, variables defined in a class can be either class variables or instance variables, and understanding the distinction between them is crucial for object-oriented programming.

Class Variables : These are the variables that are shared across all instances of a class. It is defined at the class level, outside any methods. All objects of the class share the same value for a class variable unless explicitly overridden in an object.

Instance Variables: Variables that are unique to each instance (object) of a class. These are defined within the `__init__` method or other instance methods. Each object maintains its own copy of instance variables, independent of other objects. In this example, we create a Dog class to show difference between class variables and instance variables. We also demonstrate how modifying them affects objects differently.

Code	<pre>class Dog: # Class variable species = "Canine" def __init__(self, name, age): # Instance variables self.name = name self.age = age # Create objects dog1 = Dog("Buddy", 3) dog2 = Dog("Charlie", 5) # Access class and instance variables print(dog1.species) # (Class variable) print(dog1.name) # (Instance variable) print(dog2.name) # (Instance variable) # Modify instance variables dog1.name = "Max" print(dog1.name) # (Updated instance variable) # Modify class variable Dog.species = "Feline" print(dog1.species) # (Updated class variable) print(dog2.species)</pre>
Output	<pre>Canine Buddy Charlie Max Feline Feline</pre>
Explanation	• Class Variable (species): Shared by all instances of the class. Changing <code>Dog.species</code> affects all objects, as it's a property of the class itself.

- Instance Variables (`name`, `age`): Defined in the `__init__` method. Unique to each instance (e.g., `dog1.name` and `dog2.name` are different).
- Accessing Variables: Class variables can be accessed via the class name (`Dog.species`) or an object (`dog1.species`). Instance variables are accessed via the object (`dog1.name`).
- Updating Variables: Changing `Dog.species` affects all instances. Changing `dog1.name` only affects `dog1` and does not impact `dog2`.

2.3. Inheritance

Inheritance allows a class (child class) to acquire properties and methods of another class (parent class). It supports hierarchical classification and promotes code reuse.

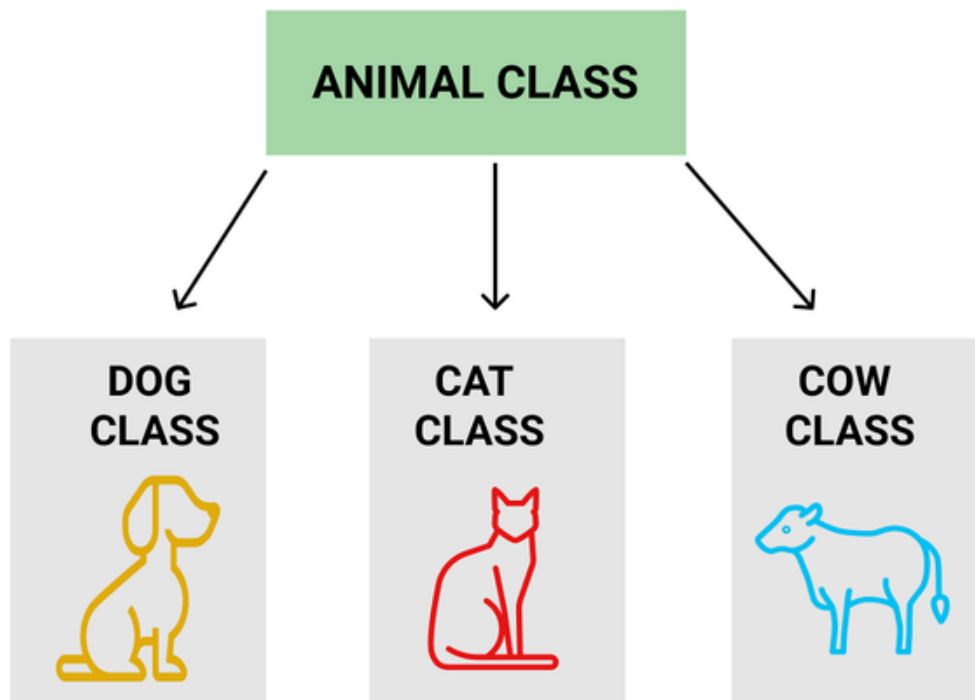


Figure 5: inheritance example

Types of Inheritance:

1. Single Inheritance: A child class inherits from a single parent class.
2. Multiple Inheritance: A child class inherits from more than one parent class.
3. Multilevel Inheritance: A child class inherits from a parent class, which in turn inherits from another class.
4. Hierarchical Inheritance: Multiple child classes inherit from a single parent class.
5. Hybrid Inheritance: A combination of two or more types of inheritance.

In this example, we create a Dog class and demonstrate single, multilevel and multiple inheritance. We show how child classes can use or extend parent class methods.

Code	<pre># Single Inheritance class Dog: def __init__(self, name): self.name = name def display_name(self): print(f"Dog's Name: {self.name}") class Labrador(Dog): # Single Inheritance def sound(self): print("Labrador woofs") # Multilevel Inheritance class GuideDog(Labrador): # Multilevel Inheritance def guide(self): print(f"{self.name}Guides the way!") # Multiple Inheritance class Friendly: def greet(self): print("Friendly!") class GoldenRetriever(Dog, Friendly): # Multiple Inheritance def sound(self): print("Golden Retriever Barks") # Example Usage lab = Labrador("Buddy") lab.display_name() lab.sound() guide_dog = GuideDog("Max") guide_dog.display_name() guide_dog.guide() retriever = GoldenRetriever("Charlie") retriever.display_name() retriever.greet() retriever.sound()</pre>
Output	<pre>Dog's Name: Buddy Labrador woofs Dog's Name: Max MaxGuides the way! Dog's Name: Charlie Friendly! Golden Retriever Barks</pre>
Explanation	• Single Inheritance: Labrador inherits Dog's attributes and methods. • Multilevel Inheritance: GuideDog extends Labrador, inheriting both Dog and Labrador functionalities. • Multiple Inheritance: GoldenRetriever inherits from both Dog and Friendly.

2.4. Polymorphism

Polymorphism in Python means "same operation, different behavior." It allows functions or methods with the same name to work differently depending on the type of object they are acting upon.

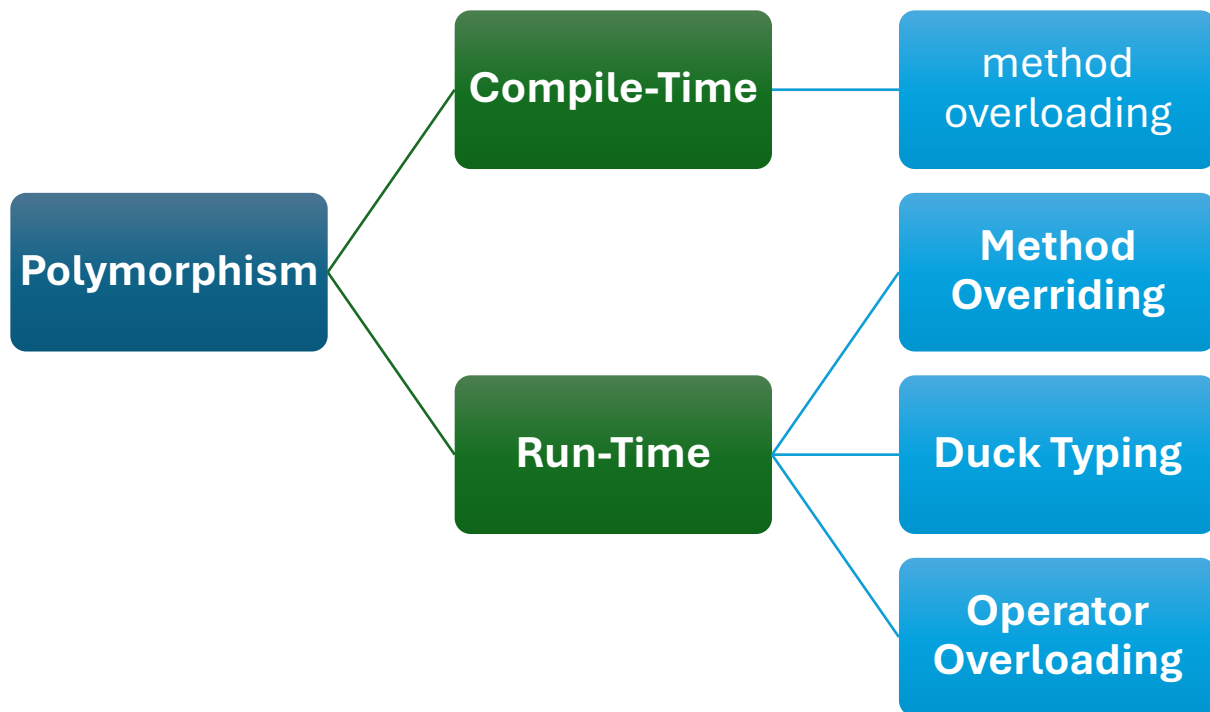


Figure 6: Polymorphism in Python

Types of Polymorphism

1- Compile-Time Polymorphism:

This type of polymorphism is determined during the compilation of the program. It allows methods or operators with the same name to behave differently based on their input parameters or usage. In languages like Java or C++, compile-time polymorphism is achieved through method overloading but it's not directly supported in Python.

In Python:

- True compile-time polymorphism is not supported.
- Instead, Python mimics it using default arguments or `*args/**kwargs`.
- Operator overloading can also be seen as part of polymorphism, though it is implemented at runtime in Python.

Code	<pre> class Calculator: def add(self, *args): return sum(args) calc = Calculator() print(calc.add(5, 10)) # Two arguments print(calc.add(5, 10, 15)) # Three arguments print(calc.add(1, 2, 3, 4)) # Any number of arguments </pre>
Output	<pre> 15 30 10 </pre>
Explanation	The <code>add</code> method uses <code>*args</code> to accept any number of positional arguments. The <code>sum(args)</code> function then calculates the sum of all the elements in the <code>args</code> tuple, which holds all the values passed to the method. <code>calc.add(5, 10)</code>: <code>args</code> becomes <code>(5, 10)</code>, and <code>sum(args)</code> returns 15. <code>calc.add(5, 10, 15)</code>: <code>args</code> becomes <code>(5, 10, 15)</code>, and <code>sum(args)</code> returns 30. <code>calc.add(1, 2, 3, 4)</code>: <code>args</code> becomes <code>(1, 2, 3, 4)</code>, and <code>sum(args)</code> returns 10.

2- Run-Time Polymorphism

Run-Time Polymorphism is determined during the execution of the program. It covers multiple forms in Python:

- **Method Overriding:** A subclass redefines a method from its parent class.
- **Duck Typing:** If an object implements the required method, it works regardless of its type.
- **Operator Overloading:** Special methods (`__add__`, `__sub__`, etc.) redefine how operators behave for user-defined objects.

In this example, we show run-time polymorphism using method overriding with dog classes and compile-time polymorphism by mimicking method overloading in a calculator class.

Code	<pre> # Method Overriding # We start with a base class and then a subclass that "overrides" the speak method. class Animal: def speak(self): return "I am an animal." class Dog(Animal): def speak(self): return "Woof!" print(Dog().speak()) # 2 Duck Typing class Cat: def speak(self): return "Meow!" def make_animal_speak(animal): # This function works for both Dog and Cat because they both have a 'speak' method. return animal.speak() print(make_animal_speak(Cat())) print(make_animal_speak(Dog())) # 3 Operator Overloading # We create a simple class that customizes the '+' operator. class Vector: def __init__(self, x, y): self.x = x self.y = y def __add__(self, other): # This special method defines the behavior of the '+' operator. return Vector(self.x + other.x, self.y + other.y) def __repr__(self): return f"Vector({self.x}, {self.y})" v1 = Vector(2, 3) v2 = Vector(4, 5) v3 = v1 + v2 print(v3) </pre>
Output	<pre> Woof! Meow! Woof! Vector(6, 8) </pre>
Explanation	• Method Overriding: Dog class overrides the speak method from the parent Animal class. This allows the program to call the speak method on a Dog object and get a specialized "Woof!" response, which is determined at runtime.

- **Duck Typing:** "make_animal_speak" function can accept both a Dog and a Cat object. It doesn't care about their class hierarchy; it only checks if they have a speak method, showcasing Python's flexible typing.
- **Operator Overloading:** "__add__" method within the Vector class is a special method that defines how the + operator behaves for Vector objects. This allows the user to add two vectors using the familiar + symbol, which is a form of syntactic sugar.

2.5.Encapsulation

Encapsulation is the bundling of data (attributes) and methods (functions) within a class, restricting access to some components to control interactions. A class is an example of encapsulation as it encapsulates all the data that is member functions, variables, etc.

Types of Encapsulation:

1. **Public Members:** Accessible from anywhere.
2. **Protected Members:** Accessible within the class and its subclasses.
3. **Private Members:** Accessible only within the class.

In this example, we create a Dog class with public, protected and private attributes. We also show how to access and modify private members using getter and setter methods.

Code

```
class Dog:
    def __init__(self, name, breed, age):
        self.name = name # Public attribute
        self._breed = breed # Protected attribute
        self.__age = age # Private attribute
    # Public method
    def get_info(self):
        return f"Name: {self.name}, Breed: {self._breed}, Age: {self.__age}"
    # Getter and Setter for private attribute
    def get_age(self):
        return self.__age
    def set_age(self, age):
        if age > 0:
            self.__age = age
        else:
            print("Invalid age!")
    # Example Usage
    dog = Dog("Buddy", "Labrador", 3)
    # Accessing public member
    print(dog.name) # Accessible
    # Accessing protected member
    print(dog._breed) # Accessible but discouraged outside the class
```

	<pre># Accessing private member using getter print(dog.get_age()) # Modifying private member using setter dog.set_age(5) print(dog.get_info())</pre>
Output	<pre>Buddy Labrador 3 Name: Buddy, Breed: Labrador, Age: 5</pre>
Explanation	• Public Members: Easily accessible, such as name. • Protected Members: Used with a single <code>_</code>, such as <code>_breed</code>. Access is discouraged but allowed in subclasses. • Private Members: Used with <code>__</code>, such as <code>__age</code>. Access requires getter and setter methods.

2.6.Data Abstraction

Abstraction hides the internal implementation details while exposing only the necessary functionality. It helps focus on "what to do" rather than "how to do it."

Types of Abstraction:

- **Partial Abstraction:** Abstract class contains both abstract and concrete methods.
- **Full Abstraction:** Abstract class contains only abstract methods (like interfaces).

In this example, we create an abstract Dog class with an abstract method (sound) and a concrete method. Subclasses implement the abstract method while inheriting the concrete method.

Code	<pre>from abc import ABC, abstractmethod class Dog(ABC): # Abstract Class def __init__(self, name): self.name = name @abstractmethod def sound(self): # Abstract Method pass def display_name(self): # Concrete Method print(f"Dog's Name: {self.name}") class Labrador(Dog): # Partial Abstraction def sound(self): print("Labrador Woof!") class Beagle(Dog): # Partial Abstraction def sound(self): print("Beagle Bark!") # Example Usage dogs = [Labrador("Buddy"), Beagle("Charlie")] for dog in dogs: dog.display_name() # Calls concrete method</pre>
-------------	--

	<code>dog.sound()</code> # Calls implemented abstract method
Output	Dog's Name: Buddy Labrador Woof! Dog's Name: Charlie Beagle Bark!
Explanation	• Partial Abstraction: The Dog class has both abstract (sound) and concrete (display_name) methods. • Why Use It: Abstraction ensures consistency in derived classes by enforcing the implementation of abstract methods.